

Preliminares matemáticos y análisis de error

Introducción

Al comenzar los cursos de química, estudiamos la *ley del gas ideal*,

$$PV = NRT,$$

que relaciona la presión P , el volumen V , la temperatura T y el número de moles N de un gas “ideal”. En esta ecuación, R es una constante que depende del sistema de medición.

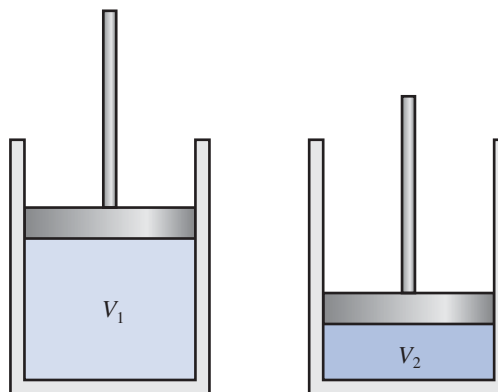
Suponga que se realizan dos experimentos para evaluar esta ley, mediante el mismo gas en cada caso. En el primer experimento,

$$\begin{aligned} P &= 1.00 \text{ atm}, & V &= 0.100 \text{ m}^3, \\ N &= 0.00420 \text{ mol}, & R &= 0.08206. \end{aligned}$$

La ley del gas ideal predice que la temperatura del gas es

$$T = \frac{PV}{NR} = \frac{(1.00)(0.100)}{(0.00420)(0.08206)} = 290.15 \text{ K} = 17^\circ\text{C}.$$

Sin embargo, cuando medimos la temperatura del gas, encontramos que la verdadera temperatura es 15°C .



A continuación, repetimos el experimento utilizando los mismos valores de R y N , pero incrementamos la presión en un factor de dos y reducimos el volumen en ese mismo factor. El producto PV sigue siendo el mismo, por lo que la temperatura prevista sigue siendo 17°C . Sin embargo, ahora encontramos que la temperatura real del gas es 19°C .

Claramente, se sospecha la ley de gas ideal, pero antes de concluir que la ley es inválida en esta situación, deberíamos examinar los datos para observar si el error se puede atribuir a los resultados del experimento. En este caso, podríamos determinar qué tan precisos deberían ser nuestros resultados experimentales para evitar que se presente un error de esta magnitud.

El análisis del error involucrado en los cálculos es un tema importante en análisis numérico y se presenta en la sección 1.2. Esta aplicación particular se considera en el ejercicio 26 de esa sección.

Este capítulo contiene una revisión breve de los temas del cálculo de una sola variable que se necesitarán en capítulos posteriores. Un conocimiento sólido de cálculo es fundamental para comprender el análisis de las técnicas numéricas y sería preciso efectuar una revisión más rigurosa para quienes no han estado en contacto con este tema durante un tiempo. Además, existe una introducción a la convergencia, el análisis de error, la representación de números en lenguaje de máquina y algunas técnicas para clasificar y minimizar el error computacional.

1.1 Revisión de cálculo

Límites y continuidad

Los conceptos de *límite* y *continuidad* de una función son fundamentales para el estudio del cálculo y constituyen la base para el análisis de las técnicas numéricas.

Definición 1.1 Una función f definida en un conjunto X de números reales que tiene el **límite** L a x_0 , escrita como

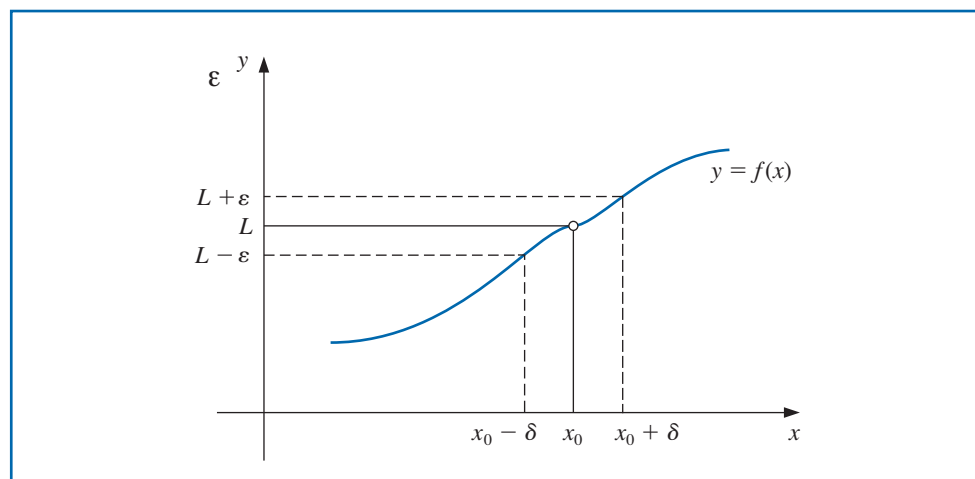
$$\lim_{x \rightarrow x_0} f(x) = L,$$

si, dado cualquier número real $\varepsilon > 0$, existe un número real $\delta > 0$, de tal forma que

$$|f(x) - L| < \varepsilon, \quad \text{siempre que } x \in X \text{ y } 0 < |x - x_0| < \delta.$$

(consulte la figura 1.1) ■

Figura 1.1



Definición 1.2

Los conceptos básicos de cálculo y sus aplicaciones se desarrollaron a finales del siglo xvii y a principios del xviii, pero los conceptos matemáticamente precisos de límites y continuidad se describieron hasta la época de Augustin Louis Cauchy (1789–1857), Heinrich Eduard Heine (1821–1881) y Karl Weierstrass (1815–1897) a finales del siglo xix.

Sea f una función definida en un conjunto X de números reales y $x_0 \in X$. Entonces f es **continua** en x_0 si

$$\lim_{x \rightarrow x_0} f(x) = f(x_0).$$

La función f es **continua en el conjunto X** si es continua en cada número en X . ■

El conjunto de todas las funciones que son continuas en el conjunto X se denota como $C(X)$. Cuando X es un intervalo de la recta real, se omiten los paréntesis en esta notación. Por ejemplo, el conjunto de todas las funciones continuas en el intervalo cerrado $[a, b]$ se denota como $C[a, b]$. El símbolo $\mathbb{R}$ denota el conjunto de todos los números reales, que también tiene la notación del intervalo $(-\infty, \infty)$. Por eso el conjunto de todas las funciones que son continuas en cada número real se denota mediante $C(\mathbb{R})$ o mediante $C(-\infty, \infty)$.

El *límite de una sucesión* de números reales o complejos se define de manera similar.

Definición 1.3

Sea $\{x_n\}_{n=1}^{\infty}$ una sucesión infinita de números reales. Esta sucesión tiene el **límite x (converge a x)** si, para cualquier $\varepsilon > 0$, existe un entero positivo $N(\varepsilon)$ tal que $|x_n - x| < \varepsilon$ siempre que $n > N(\varepsilon)$. La notación

$$\lim_{n \rightarrow \infty} x_n = x, \quad \text{o} \quad x_n \rightarrow x \quad \text{en} \quad n \rightarrow \infty,$$

significa que la sucesión $\{x_n\}_{n=1}^{\infty}$ converge a x . ■

Teorema 1.4

Si f es una función definida en un conjunto X de números reales y $x_0 \in X$, entonces los siguientes enunciados son equivalentes:

- f es continua en x_0 ;
- Si $\{x_n\}_{n=1}^{\infty}$ es cualquier sucesión en X , que converge a x_0 , entonces $\lim_{n \rightarrow \infty} f(x_n) = f(x_0)$. ■

Se asumirá que las funciones que consideraremos al analizar los métodos numéricos son continuas porque éste es el requisito mínimo para una conducta predecible. Las funciones que no son continuas pueden pasar por alto puntos de interés, lo cual puede causar dificultades al intentar aproximar la solución de un problema.

Diferenciabilidad

Las suposiciones más sofisticadas sobre una función por lo general conducen a mejores resultados de aproximación. Por ejemplo, normalmente una función con una gráfica suave se comportaría de forma más predecible que una con numerosas características irregulares. La condición de uniformidad depende del concepto de la derivada.

Definición 1.5

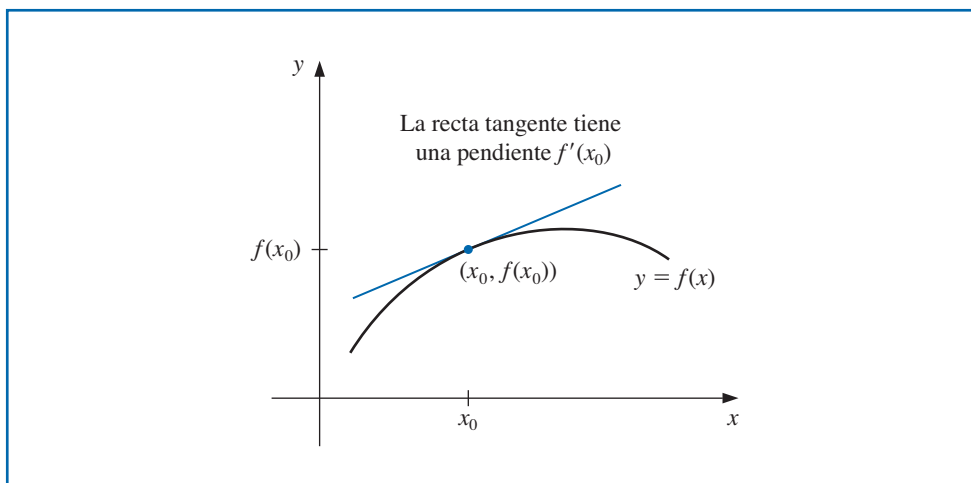
Si f es una función definida en un intervalo abierto que contiene x_0 . La función f es **diferenciable** en x_0 si

$$f'(x_0) = \lim_{x \rightarrow x_0} \frac{f(x) - f(x_0)}{x - x_0}$$

existe. El número $f'(x_0)$ recibe el nombre de **derivada** de f en x_0 . Una función que tiene una derivada en cada número en un conjunto X es **diferenciable en X** . ■

La derivada de f en x_0 es la pendiente de la recta tangente a la gráfica de f en $(x_0, f(x_0))$, como se muestra en la figura 1.2.

Figura 1.2



Teorema 1.6 Si la función f es diferenciable en x_0 , entonces f es continua en x_0 . ■

El teorema atribuido a Michel Rolle (1652–1719) apareció en 1691 en un tratado poco conocido titulado *Méthode pour résoudre les égalités* (Método para resolver las igualdades). Originalmente, Rolle criticaba el cálculo desarrollado por Isaac Newton y Gottfried Leibniz, pero después se convirtió en uno de sus defensores.

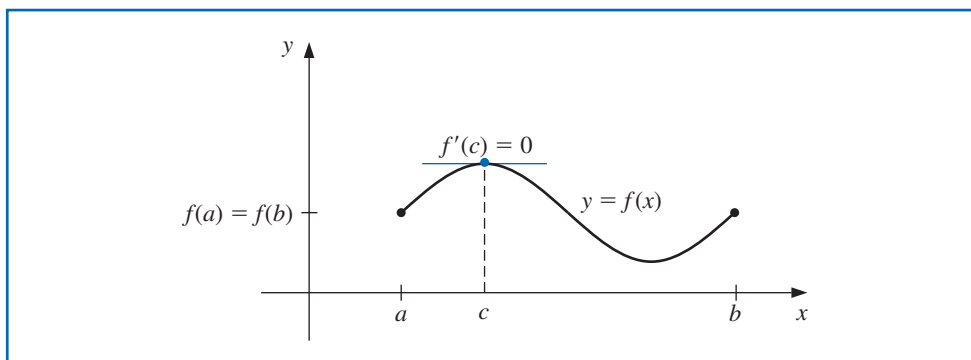
Los siguientes teoremas son de importancia fundamental al deducir los métodos para estimación del cálculo de error. Las pruebas de estos teoremas y los otros resultados sin referencias en esta sección se pueden encontrar en cualquier texto de cálculo estándar.

El conjunto de todas las funciones que tienen derivadas continuas n en X se denota como $C^n(X)$ y el conjunto de funciones que tienen derivadas de todos los órdenes en X se denota como $C^\infty(X)$. Las funciones polinomial, racional, trigonométrica, exponencial y logarítmica se encuentran en $C^\infty(X)$, donde X consiste en todos los números para los que se definen las funciones. Cuando X es un intervalo de la recta real, de nuevo se omiten los paréntesis en esta notación.

Teorema 1.7 (Teorema de Rolle)

Suponga que $f \in C[a, b]$ y f es diferenciable en (a, b) . Si $f(a) = f(b)$, entonces existe un número c en (a, b) con $f'(c) = 0$. (Consulte la figura 1.3.) ■

Figura 1.3

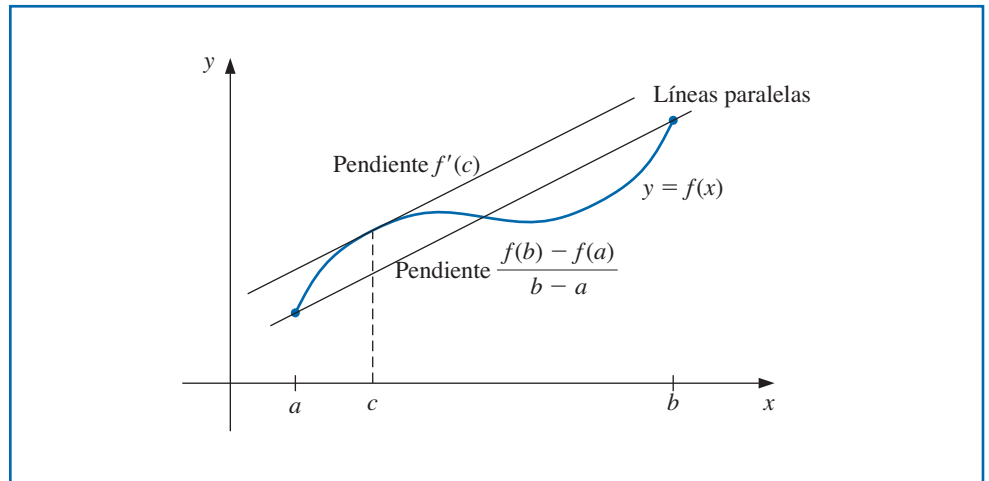


Teorema 1.8 (Teorema del valor medio)

Si $f \in C[a, b]$ y f es diferenciable en (a, b) , entonces existe un número c en (a, b) con (consulte la figura 1.4.)

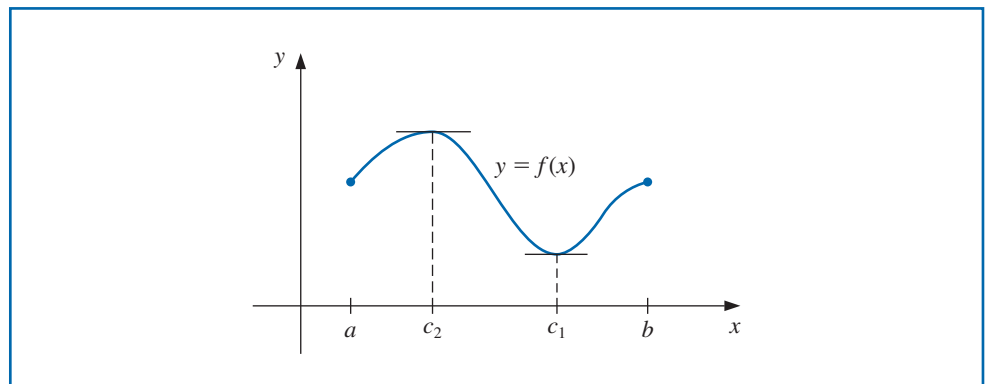
$$f'(c) = \frac{f(b) - f(a)}{b - a}.$$

Figura 1.4

**Teorema 1.9 (Teorema del valor extremo)**

Si $f \in C[a, b]$, entonces existe $c_1, c_2 \in [a, b]$ con $f(c_1) \leq f(x) \leq f(c_2)$, para todas las $x \in [a, b]$. Además, si f es diferenciable en (a, b) , entonces se presentan los números c_1 y c_2 ya sea en los extremos de $[a, b]$ o donde f' es cero. (Consulte la figura 1.5.) ■

Figura 1.5



Ejemplo 1 Encuentre los valores mínimo absoluto y máximo absoluto de

$$f(x) = 2 - e^x + 2x$$

en los intervalos **a)** $[0, 1]$ y **b)** $[1, 2]$.

Solución Comenzamos por derivar $f(x)$ para obtener

$$f'(x) = -e^x + 2.$$

$f'(x) = 0$ cuando $-e^x + 2 = 0$ o de forma equivalente, cuando $e^x = 2$. Al tomar el logaritmo natural de ambos lados de la ecuación obtenemos

$$\ln(e^x) = \ln(2) \text{ o } x = \ln(2) \approx 0.69314718056$$

- a) Cuando el intervalo es $[0, 1]$, el extremo absoluto debe ocurrir en $f(0)$, $f(\ln(2))$, o $f(1)$. Al evaluar, tenemos

$$f(0) = 2 - e^0 + 2(0) = 1$$

$$f(\ln(2)) = 2 - e^{\ln(2)} + 2 \ln(2) = 2 \ln(2) \approx 1.38629436112$$

$$f(1) = 2 - e + 2(1) = 4 - e \approx 1.28171817154.$$

Por lo tanto, el mínimo absoluto de $f(x)$ en $[0, 1]$ es $f(0) = 1$ y el máximo absoluto es $f(\ln(2)) = 2 \ln(2)$.

- b) Cuando el intervalo es $[1, 2]$, sabemos que $f'(x) \neq 0$, por lo que el extremo absoluto se presenta en $f(1)$ y $f(2)$. Por lo tanto, $f(2) = 2 - e^2 + 2(2) = 6 - e^2 \approx -1.3890560983$

El mínimo absoluto en $[1, 2]$ es $6 - e^2$ y el máximo absoluto es 1.

Observamos que

$$\max_{0 \leq x \leq 2} |f(x)| = |6 - e^2| \approx 1.3890560983. \quad \blacksquare$$

En general, el siguiente teorema no se presenta en un curso de cálculo básico, pero se deriva al aplicar el teorema de Rolle sucesivamente a f , f' , $\dots$, y finalmente, a $f^{(n-1)}$. Este resultado se considera en el ejercicio 26.

Teorema 1.10 (Teorema generalizado de Rolle)

Suponga que $f \in C[a, b]$ es n veces diferenciable en (a, b) . Si $f(x) = 0$ en los $n + 1$ números distintos $a \leq x_0 < x_1 < \dots < x_n \leq b$, entonces un número c en (x_0, x_n) y, por lo tanto, en (a, b) existe con $f^{(n)}(c) = 0$. ■

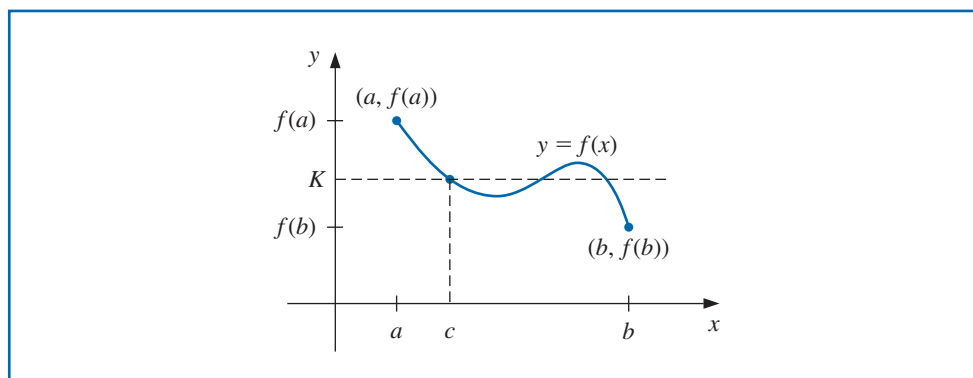
También utilizaremos con frecuencia el teorema del valor intermedio. A pesar de que esta declaración parece razonable, su prueba va más allá del alcance del curso habitual de cálculo. Sin embargo, se puede encontrar en muchos textos de análisis (consulte, por ejemplo, [Fu], p. 67).

Teorema 1.11 (Teorema del valor intermedio)

Si $f \in C[a, b]$ y K es cualquier número entre $f(a)$ y $f(b)$, entonces existe un número c en (a, b) para el cual $f(c) = K$. ■

La figura 1.6 muestra una opción para el número garantizada por el teorema del valor intermedio. En este ejemplo, existen otras dos posibilidades.

Figura 1.6



Ejemplo 2 Muestre que $x^5 - 2x^3 + 3x^2 - 1 = 0$ tiene una solución en el intervalo $[0, 1]$.

Solución Considere la función definida por $f(x) = x^5 - 2x^3 + 3x^2 - 1$. La función f es continua en $[0, 1]$. Además,

$$f(0) = -1 < 0 \quad \text{y} \quad 0 < 1 = f(1).$$

Por lo tanto, el teorema del valor intermedio implica que existe un número c , con $0 < c < 1$, para el cual $c^5 - 2c^3 + 3c^2 - 1 = 0$. ■

Como se observa en el ejemplo 2, el teorema del valor intermedio se utiliza para determinar cuándo existen soluciones para ciertos problemas. Sin embargo, no provee un medio eficiente para encontrar estas soluciones. Este tema se considera en el capítulo 2.

Integración

El otro concepto básico del cálculo que se utilizará ampliamente es la integral de Riemann.

Definición 1.12 La **integral de Riemann** de la función f en el intervalo $[a, b]$ es el siguiente límite, siempre y cuando exista:

$$\int_a^b f(x) dx = \lim_{\max \Delta x_i \rightarrow 0} \sum_{i=1}^n f(z_i) \Delta x_i,$$

donde los números $x_0, x_1, \dots, x_n$ satisfacen $a = x_0 \leq x_1 \leq \dots \leq x_n = b$, donde $\Delta x_i = x_i - x_{i-1}$, para cada $i = 1, 2, \dots, n$, y z_i se selecciona de manera arbitraria en el intervalo $[x_{i-1}, x_i]$. ■

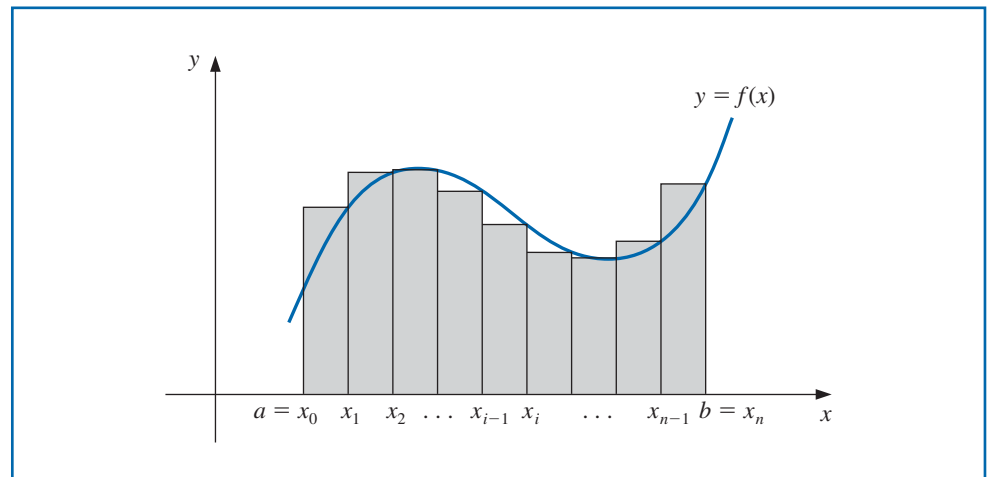
Una función f que es continua en un intervalo $[a, b]$ es también Riemann integrable en $[a, b]$. Esto nos permite elegir, para conveniencia computacional, los puntos x_i se separarán equitativamente en $[a, b]$ para cada $i = 1, 2, \dots, n$, para seleccionar $z_i = x_i$. En este caso,

$$\int_a^b f(x) dx = \lim_{n \rightarrow \infty} \frac{b-a}{n} \sum_{i=1}^n f(x_i),$$

donde los números mostrados en la figura 1.7, como x_i , son $x_i = a + i(b-a)/n$.

George Fredrich Bernhard Riemann (1826–1866) realizó muchos de los descubrimientos importantes para clasificar las funciones que tienen integrales. También realizó trabajos fundamentales en geometría y la teoría de funciones complejas y se le considera uno de los matemáticos prolíferos del siglo XIX.

Figura 1.7



Se necesitarán otros dos resultados en nuestro estudio para análisis numérico. El primero es una generalización del teorema del valor promedio para integrales.

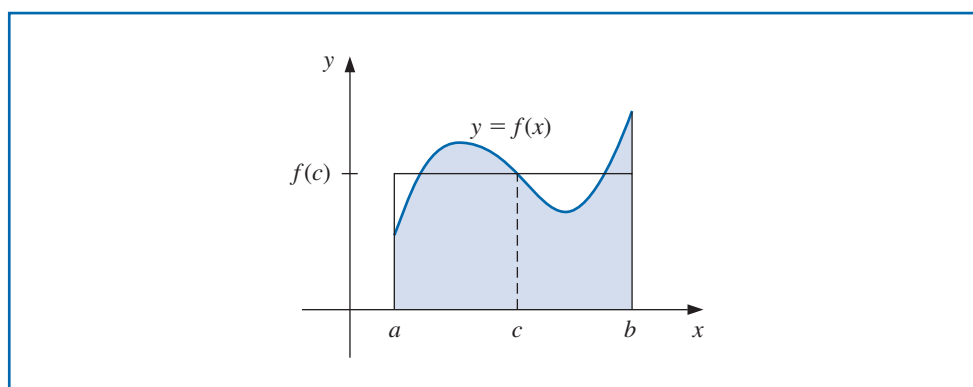
Teorema 1.13 (Teorema del valor promedio para integrales)

Suponga que $f \in C[a, b]$, la integral de Riemann de g existe en $[a, b]$, y $g(x)$ no cambia de signo en $[a, b]$. Entonces existe un número c en (a, b) con

$$\int_a^b f(x)g(x) dx = f(c) \int_a^b g(x) dx. \quad \blacksquare$$

Cuando $g(x) \equiv 1$, el teorema 1.13 es el teorema del valor medio para integrales. Éste proporciona el **valor promedio** de la función f sobre el intervalo $[a, b]$ como (consulte la figura 1.8.)

$$f(c) = \frac{1}{b-a} \int_a^b f(x) dx.$$

Figura 1.8

En general la prueba del teorema 1.13 no se da en un curso básico de cálculo, pero se puede encontrar en muchos textos de análisis (consulte, por ejemplo, [Fu], p. 162)

Polinomios y series de Taylor

El teorema final en esta revisión de cálculo describe los polinomios de Taylor. Estos polinomios se usan ampliamente en el análisis numérico.

Teorema 1.14 (Teorema de Taylor)

Brook Taylor (1685–1731) describió esta serie en 1715 en el artículo *Methodus incrementorum directa et inversa* (*Métodos para incrementos directos e inversos*). Isaac Newton, James Gregory y otros ya conocían algunos casos especiales del resultado y, probablemente, el resultado mismo.

Suponga que $f \in C^n[a, b]$, $f^{(n+1)}$ existe en $[a, b]$, y $x_0 \in [a, b]$. Para cada $x \in [a, b]$, existe un número $\xi(x)$ entre x_0 y x con

$$f(x) = P_n(x) + R_n(x),$$

donde

$$\begin{aligned} P_n(x) &= f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2!}(x - x_0)^2 + \cdots + \frac{f^{(n)}(x_0)}{n!}(x - x_0)^n \\ &= \sum_{k=0}^n \frac{f^{(k)}(x_0)}{k!}(x - x_0)^k \end{aligned}$$

y

$$R_n(x) = \frac{f^{(n+1)}(\xi(x))}{(n+1)!} (x - x_0)^{n+1}.$$

Colin Maclaurin (1698-1746) es más conocido como el defensor del cálculo de Newton cuando éste fue objeto de los ataques implacables del obispo y filósofo irlandés George Berkeley.

Maclaurin no descubrió la serie que lleva su nombre; los matemáticos del siglo ya la conocían desde antes de que él naciera. Sin embargo, concibió un método para resolver un sistema de ecuaciones lineales que se conoce como regla de Cramer, que Cramer no publicó hasta 1750.

Aquí $P_n(x)$ es llamado el **n -ésimo polinomio de Taylor** para f alrededor de x_0 y $R_n(x)$ recibe el nombre de **residuo** (o **error de truncamiento**) relacionado con $P_n(x)$. Puesto que el número $\xi(x)$ en el error de truncamiento $R_n(x)$ depende del valor de x donde se evalúa el polinomio $P_n(x)$, es una función de la variable x . Sin embargo, no deberíamos esperar ser capaces de determinar la función $\xi(x)$ de manera explícita. El teorema de Taylor simplemente garantiza que esta función existe y que su valor se encuentra entre x y x_0 . De hecho, uno de los problemas comunes en los métodos numéricos es tratar de determinar un límite realista para el valor de $f^{(n+1)}(\xi(x))$ cuando x se encuentra en un intervalo específico.

La serie infinita obtenida al tomar el límite de $P_n(x)$ conforme $n \rightarrow \infty$ recibe el nombre de **serie de Taylor** para f alrededor de x_0 . En caso de que $x_0 = 0$, entonces al polinomio de Taylor con frecuencia se le llama **polinomio de Maclaurin** y a la serie de Taylor a menudo se le conoce como **serie de Maclaurin**.

El término **error de truncamiento** en el polinomio de Taylor se refiere al error implicado al utilizar una suma truncada, o finita, para aproximar la suma de una serie infinita.

Ejemplo 3 Si $f(x) = \cos x$ y $x_0 = 0$. Determine

- a) el segundo polinomio de Taylor para f alrededor de x_0 ; y
- b) el tercer polinomio de Taylor para f alrededor de x_0 .

Solución Puesto que $f \in C^\infty(\mathbb{R})$, el teorema de Taylor puede aplicarse a cualquiera $n \geq 0$. Además,

$$f'(x) = -\sin x, \quad f''(x) = -\cos x, \quad f'''(x) = \sin x, \quad \text{y} \quad f^{(4)}(x) = \cos x,$$

Por lo tanto

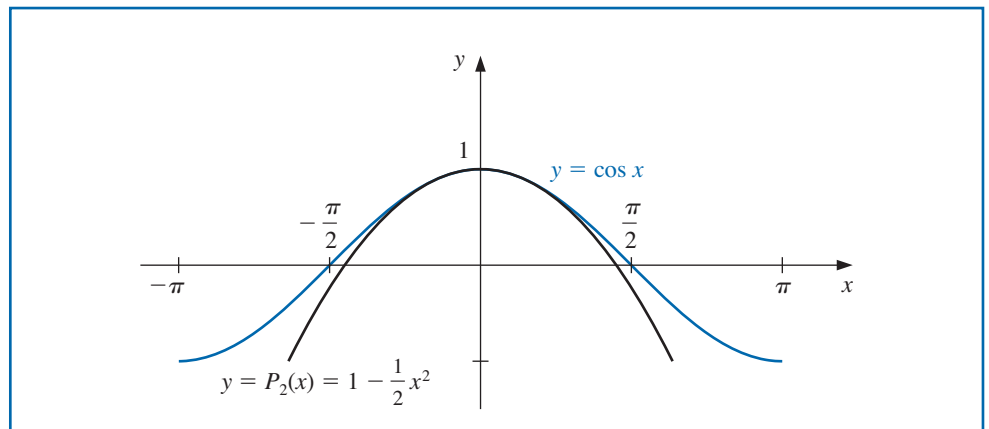
$$f(0) = 1, \quad f'(0) = 0, \quad f''(0) = -1, \quad \text{y} \quad f'''(0) = 0.$$

- a) Para $n = 2$ y $x_0 = 0$, obtenemos

$$\begin{aligned} \cos x &= f(0) + f'(0)x + \frac{f''(0)}{2!}x^2 + \frac{f'''(\xi(x))}{3!}x^3 \\ &= 1 - \frac{1}{2}x^2 + \frac{1}{6}x^3 \sin \xi(x), \end{aligned}$$

donde $\xi(x)$ es algún número (por lo general, desconocido) entre 0 y x . (Consulte la figura 1.9.)

Figura 1.9



Cuando $x = 0.01$, esto se convierte en

$$\cos 0.01 = 1 - \frac{1}{2}(0.01)^2 + \frac{1}{6}(0.01)^3 \sin \xi(0.01) = 0.99995 + \frac{10^{-6}}{6} \sin \xi(0.01).$$

Por lo tanto, la aproximación para $\cos 0.01$ provista por el polinomio de Taylor es 0.99995. El error de truncamiento, o término restante, relacionado con esta aproximación es

$$\frac{10^{-6}}{6} \sin \xi(0.01) = 0.1\bar{6} \times 10^{-6} \sin \xi(0.01),$$

donde la barra sobre el 6 en $0.1\bar{6}$ se utiliza para indicar que este dígito se repite indefinidamente. A pesar de que no existe una forma de determinar $\sin \xi(0.01)$, sabemos que todos los valores del seno se encuentran en el intervalo $[-1, 1]$, por lo que el error que se presenta si utilizamos la aproximación 0.99995 para el valor de $\cos 0.01$ está limitado por

$$|\cos(0.01) - 0.99995| = 0.1\bar{6} \times 10^{-6} |\sin \xi(0.01)| \leq 0.1\bar{6} \times 10^{-6}.$$

Por lo tanto, la aproximación 0.99995 corresponde por lo menos a los primeros cinco dígitos de $\cos 0.01$ y

$$\begin{aligned} 0.9999483 < 0.99995 - 1.6 \times 10^{-6} &\leq \cos 0.01 \\ &\leq 0.99995 + 1.6 \times 10^{-6} < 0.9999517. \end{aligned}$$

El límite del error es mucho más grande que el error real. Esto se debe, en parte, al escaso límite que usamos para $|\sin \xi(x)|$. En el ejercicio 27 se muestra que para todos los valores de x , tenemos $|\sin x| \leq |x|$. Puesto que $0 \leq \xi < 0.01$, podríamos haber usado el hecho de que $|\sin \xi(x)| \leq 0.01$ en la fórmula de error, lo cual produce el límite $0.1\bar{6} \times 10^{-8}$.

b) Puesto que $f'''(0) = 0$, el tercer polinomio de Taylor con el término restante alrededor de $x_0 = 0$ es

$$\cos x = 1 - \frac{1}{2}x^2 + \frac{1}{24}x^4 \cos \tilde{\xi}(x),$$

donde $0 < \tilde{\xi}(x) < 0.01$. El polinomio de aproximación sigue siendo el mismo y la aproximación sigue siendo 0.99995, pero ahora tenemos mayor precisión. Puesto que $|\cos \tilde{\xi}(x)| \leq 1$ para todas las x , obtenemos

$$\left| \frac{1}{24}x^4 \cos \tilde{\xi}(x) \right| \leq \frac{1}{24}(0.01)^4(1) \approx 4.2 \times 10^{-10}.$$

por lo tanto

$$|\cos 0.01 - 0.99995| \leq 4.2 \times 10^{-10},$$

y

$$\begin{aligned} 0.99994999958 &= 0.99995 - 4.2 \times 10^{-10} \\ &\leq \cos 0.01 \leq 0.99995 + 4.2 \times 10^{-10} = 0.99995000042. \end{aligned} \quad \blacksquare$$

El ejemplo 3 ilustra los dos objetivos del análisis numérico:

- i) Encuentre una aproximación a la solución de un problema determinado.
- ii) Determine un límite o cota para la precisión de la aproximación.

Los polinomios de Taylor en ambas partes proporcionan la misma respuesta para i), pero el tercero provee una respuesta mucho mejor para ii) que el segundo. También podemos utilizar estos polinomios para obtener aproximaciones de las integrales.

Ilustración Podemos utilizar el tercer polinomio de Taylor y su término restante encontrado en el ejemplo 3 para aproximar $\int_0^{0.1} \cos x \, dx$. Tenemos

$$\begin{aligned} \int_0^{0.1} \cos x \, dx &= \int_0^{0.1} \left(1 - \frac{1}{2}x^2\right) dx + \frac{1}{24} \int_0^{0.1} x^4 \cos \tilde{\xi}(x) \, dx \\ &= \left[x - \frac{1}{6}x^3\right]_0^{0.1} + \frac{1}{24} \int_0^{0.1} x^4 \cos \tilde{\xi}(x) \, dx \\ &= 0.1 - \frac{1}{6}(0.1)^3 + \frac{1}{24} \int_0^{0.1} x^4 \cos \tilde{\xi}(x) \, dx. \end{aligned}$$

Por lo tanto,

$$\int_0^{0.1} \cos x \, dx \approx 0.1 - \frac{1}{6}(0.1)^3 = 0.0998\bar{3}.$$

Un límite o cota para el error en esta aproximación se determina a partir de la integral del término restante de Taylor y el hecho de que $|\cos \tilde{\xi}(x)| \leq 1$ para todas las x :

$$\begin{aligned} \frac{1}{24} \left| \int_0^{0.1} x^4 \cos \tilde{\xi}(x) \, dx \right| &\leq \frac{1}{24} \int_0^{0.1} x^4 |\cos \tilde{\xi}(x)| \, dx \\ &\leq \frac{1}{24} \int_0^{0.1} x^4 \, dx = \frac{(0.1)^5}{120} = 8.\bar{3} \times 10^{-8}. \end{aligned}$$

El valor verdadero de esta integral es

$$\int_0^{0.1} \cos x \, dx = \left[\sin x \right]_0^{0.1} = \sin 0.1 \approx 0.099833416647,$$

por lo que el error real para esta aproximación es 8.3314×10^{-8} , que se encuentra dentro del límite del error. ■

La sección Conjunto de ejercicios 1.1 está disponible en línea. Encuentre la ruta de acceso en las páginas preliminares.

1.2 Errores de redondeo y aritmética computacional

La aritmética realizada con una calculadora o computadora es diferente a la aritmética que se imparte en los cursos de álgebra y cálculo. Podría esperarse que declaraciones como $2+2=4$, $4 \cdot 8=32$, y $(\sqrt{3})^2=3$ siempre sean verdaderas; sin embargo con la aritmética *computacional*, esperamos resultados exactos para $2+2=4$ y $4 \cdot 8=32$, pero no obtendremos exactamente $(\sqrt{3})^2=3$. Para comprender por qué esto es verdadero, debemos explorar el mundo de la aritmética de dígitos finitos.

En nuestro mundo matemático tradicional, permitimos números con una cantidad infinita de dígitos. La aritmética que usamos en este mundo *define* $\sqrt{3}$ como el único número positivo que cuando se multiplica por sí mismo produce el entero 3. No obstante, en el mundo computacional, cada número representable sólo tiene un número fijo y finito de dígitos. Esto significa que, por ejemplo, sólo los números racionales, e incluso no todos ellos, se pueden representar de forma exacta. Ya que $\sqrt{3}$ no es racional, se proporciona una representación aproximada, cuyo cuadrado no será exactamente 3, a pesar de que es probable que esté suficientemente cerca de 3 para ser aceptable en la mayoría de las situaciones. Entonces, en muchos casos, esta aritmética mecánica es satisfactoria y pasa sin importancia o preocupación, pero algunas veces surgen problemas debido a su discrepancia.

Como secuencia, este número de máquina representa precisamente el número decimal

$$\begin{aligned} (-1)^s 2^{c-1023} (1+f) &= (-1)^0 \cdot 2^{1027-1023} \left(1 + \left(\frac{1}{2} + \frac{1}{8} + \frac{1}{16} + \frac{1}{32} + \frac{1}{256} + \frac{1}{4096} \right) \right) \\ &= 27.56640625. \end{aligned}$$

Sin embargo, el siguiente número de máquina más pequeño es

[illegible]

el siguiente número de máquina más grande es

[illegible]

Esto significa que nuestro número de máquina original no sólo representa 27.56640625, sino también la mitad de los números reales que se encuentran entre 27.56640625 y el siguiente número de máquina más pequeño, así como la mitad de los números entre 27.56640625 y el siguiente número de máquina más grande. Para ser preciso, representa cualquier número

[27.5664062499999982236431605997495353221893310546875,

27.5664062500000017763568394002504646778106689453125).

El número positivo normalizado más pequeño que se puede representar tiene $s = 0$, $c = 1$, y $f = 0$ y es equivalente

$$2^{-1022} \cdot (1 + 0) \approx 0.22251 \times 10^{-307}.$$

y el más grande tiene $s = 0$, $c = 2046$ y $f = 1 - 2^{-52}$ y es equivalente a

$$2^{1023} \cdot (2 - 2^{-52}) \approx 0.17977 \times 10^{309}.$$

Los números que se presentan en los cálculos que tienen una magnitud menor que

$$2^{-1022} \cdot (1 + 0)$$

resultan en un **subdesbordamiento** y, en general, se configuran en cero. Los números superiores a

$$2^{1023} \cdot (2 - 2^{-52})$$

resultan en **desbordamiento** y, comúnmente, causan que los cálculos se detengan (a menos que el programa haya sido diseñado para detectar estas presencias). Observe que existen dos representaciones para el número cero: un 0 positivo cuando $s = 0$, $c = 0$ y $f = 0$, y un 0 negativo cuando $s = 1$, $c = 0$ y $f = 0$.

Números de máquina decimales

El uso de dígitos binarios tiende a ocultar las dificultades computacionales que se presentan cuando se usa una colección finita de números de máquina para representar todos los números reales. Para examinar estos problemas, utilizaremos números decimales más familiares en lugar de una representación binaria. En específico, suponemos que los números máquina se representan en formato normalizado de punto flotante *decimal*

$$\pm 0.d_1 d_2 \dots d_k \times 10^n, \quad 1 \leq d_1 \leq 9, \quad \text{y} \quad 0 \leq d_i \leq 9,$$

para cada $i = 2, \dots, k$. Los números de esta forma reciben el nombre de *números de máquina decimales de dígito k* .

Cualquier número real positivo dentro del rango numérico de la máquina puede ser normalizado a la forma

$$y = 0.d_1d_2 \dots d_k d_{k+1}d_{k+2} \dots \times 10^n.$$

El error que resulta de reemplazar un número con esta forma de punto flotante se llama **error de redondeo**, independientemente de si se usa el método de redondeo o de corte.

La forma de punto flotante de y , que se denota $fl(y)$, se obtiene al terminar la mantisa de y en los dígitos decimales de k . Existen dos maneras comunes para realizar esta terminación. Un método, llamado **de corte**, es simplemente cortar los dígitos $d_{k+1}d_{k+2} \dots$. Esto produce la forma de punto flotante

$$fl(y) = 0.d_1d_2 \dots d_k \times 10^n.$$

El otro método, llamado **redondeo**, suma $5 \times 10^{n-(k+1)}$ a y y entonces corta el resultado para obtener un número con la forma

$$fl(y) = 0.\delta_1\delta_2 \dots \delta_k \times 10^n.$$

Para redondear, cuando $d_{k+1} \geq 5$, sumamos 1 a d_k para obtener $fl(y)$; es decir, *redondeamos hacia arriba*. Cuando $d_{k+1} < 5$, simplemente cortamos todo, excepto los primeros dígitos k ; es decir, *redondeamos hacia abajo*. Si redondeamos hacia abajo, entonces $\delta_i = d_i$, para cada $i = 1, 2, \dots, k$. Sin embargo, si redondeamos hacia arriba, los dígitos (e incluso el exponente) pueden cambiar.

Ejemplo 1 Determine los valores a) de corte y b) de redondeo de cinco dígitos del número irracional π .

Solución El número π tiene una expansión decimal infinita de la forma $\pi = 3.14159265 \dots$. Escrito en una forma decimal normalizada, tenemos

$$\pi = 0.314159265 \dots \times 10^1.$$

a) El formato de punto flotante de π usando el recorte de cinco dígitos es

$$fl(\pi) = 0.31415 \times 10^1 = 3.1415.$$

b) El sexto dígito de la expansión decimal de π es un 9, por lo que el formato de punto flotante de π con redondeo de cinco dígitos es

$$fl(\pi) = (0.31415 + 0.00001) \times 10^1 = 3.1416. \quad \blacksquare$$

La siguiente definición describe tres métodos para medir errores de aproximación.

Definición 1.15 Suponga que p^* es una aproximación a p . El **error real** es $p - p^*$, el **error absoluto** es $|p - p^*|$, y el **error relativo** es $\frac{|p - p^*|}{|p|}$, siempre y cuando $p \neq 0$. $\blacksquare$

Considere los errores real, absoluto y relativo al representar p con p^* en el siguiente ejemplo.

Ejemplo 2 Determine los errores real, absoluto y relativo al aproximar p con p^* cuando

- a) $p = 0.3000 \times 10^1$ y $p^* = 0.3100 \times 10^1$;
- b) $p = 0.3000 \times 10^{-3}$ y $p^* = 0.3100 \times 10^{-3}$;
- c) $p = 0.3000 \times 10^4$ y $p^* = 0.3100 \times 10^4$.

En general, el error relativo es una mejor medición de precisión que el error absoluto porque considera el tamaño del número que se va a aproximar.

Solución

- a) Para $p = 0.3000 \times 10^1$ y $p^* = 0.3100 \times 10^1$, el error real es -0.1 , el error absoluto es 0.1 y el error relativo es $0.333\bar{3} \times 10^{-1}$.
- b) Para $p = 0.3000 \times 10^{-3}$ y $p^* = 0.3100 \times 10^{-3}$, el error real es -0.1×10^{-4} , el error absoluto es 0.1×10^{-4} y el error relativo es $0.333\bar{3} \times 10^{-1}$.
- c) Para $p = 0.3000 \times 10^4$ y $p^* = 0.3100 \times 10^4$, el error real es -0.1×10^3 , el error absoluto es 0.1×10^3 y, de nuevo, el error relativo es $0.333\bar{3} \times 10^{-1}$.

A menudo no podemos encontrar un valor preciso para el error verdadero en una aproximación. Por el contrario, encontramos una cota para el error, lo cual nos proporciona un error del “peor caso”.

Este ejemplo muestra que el mismo error relativo, $0.333\bar{3} \times 10^{-1}$, se presenta para errores absolutos ampliamente variables. Como una medida de precisión, el error absoluto puede ser engañoso y el error relativo más significativo debido a que este error considera el tamaño del valor. ■

Un límite de error es un número no negativo mayor que el error absoluto. Algunas veces se obtiene con los métodos de cálculo para encontrar el valor absoluto máximo de una función. Esperamos encontrar el límite superior más pequeño posible para el error a fin de obtener un estimado del error real que es lo más preciso posible.

La siguiente definición usa el error relativo para proporcionar una medida de dígitos significativos de precisión para una aproximación.

Definición 1.16

A menudo, el término *dígitos significativos* se usa para describir vagamente el número de dígitos decimales que parecen ser exactos. La definición es más precisa y provee un concepto continuo.

Se dice que el número p^* se aproxima a p para t **dígitos significativos** (o cifras) si t es el entero no negativo más grande para el que

$$\frac{|p - p^*|}{|p|} \leq 5 \times 10^{-t}.$$

La tabla 1.1 ilustra la naturaleza continua de los dígitos significativos al enumerar, para los diferentes valores de p , el límite superior mínimo de $|p - p^*|$, denominado máx. $|p - p^*|$, cuando p^* concuerda con p en cuatro dígitos significativos.

Tabla 1.1

p	0.1	0.5	100	1000	5000	9990	10000
máx $ p - p^* $	0.00005	0.00025	0.05	0.5	2.5	4.995	5.

Al regresar a la representación de los números de máquina, observamos que la representación de punto flotante $fl(y)$ para el número y tiene el error relativo

$$\left| \frac{y - fl(y)}{y} \right|.$$

Si se usan k dígitos decimales y corte para la representación de máquina de

$$y = 0.d_1d_2 \dots d_k d_{k+1} \dots \times 10^n,$$

entonces

$$\begin{aligned} \left| \frac{y - fl(y)}{y} \right| &= \left| \frac{0.d_1d_2 \dots d_k d_{k+1} \dots \times 10^n - 0.d_1d_2 \dots d_k \times 10^n}{0.d_1d_2 \dots \times 10^n} \right| \\ &= \left| \frac{0.d_{k+1}d_{k+2} \dots \times 10^{n-k}}{0.d_1d_2 \dots \times 10^n} \right| = \left| \frac{0.d_{k+1}d_{k+2} \dots}{0.d_1d_2 \dots} \right| \times 10^{-k}. \end{aligned}$$

Puesto que $d_1 \neq 0$, el valor mínimo del denominador es 0.1. El numerador se limita en la parte superior mediante 1. Como consecuencia,

$$\left| \frac{y - fl(y)}{y} \right| \leq \frac{1}{0.1} \times 10^{-k} = 10^{-k+1}.$$

De igual forma, un límite para el error relativo al utilizar aritmética de redondeo de dígitos k es $0.5 \times 10^{-k+1}$. (Consulte el ejercicio 28.)

Observe que los límites para el error relativo mediante aritmética de dígitos k son independientes del número que se va a representar. Este resultado se debe a la forma en la que se distribuyen los números de máquina a lo largo de la recta real. Debido al formato exponencial de la característica, el mismo número de los números de máquina decimales se usa para representar cada uno de los intervalos $[0.1, 1]$, $[1, 10]$ y $[10, 100]$. De hecho, dentro de los límites de la máquina, el número de los números de máquina decimales en $[10^n, 10^{n+1}]$ es constante para todos los enteros n .

Aritmética de dígitos finitos

Además de la representación inexacta de números, la aritmética que se efectúa en una computadora no es exacta. La aritmética implica manipular dígitos binarios mediante diferentes operaciones de cambio, o lógicas. Puesto que la mecánica real de estas operaciones no es pertinente para esta presentación, debemos idear una aproximación propia para aritmética computacional. A pesar de que nuestra aritmética no proporcionará el panorama exacto, es suficiente para explicar los problemas que se presentan. (Para una explicación de las manipulaciones realmente incluidas, se insta al lector a consultar textos de ciencias computacionales con una orientación más técnica, como [Ma], *Computer System Architecture* [Arquitectura de sistemas computacionales].)

Piense que las representaciones de punto flotante $fl(x)$ y $fl(y)$ están dadas para los números reales x y y y que los símbolos $\oplus$, $\ominus$, $\otimes$, y $\oslash$ representan operaciones de máquina de suma, resta, multiplicación y división, respectivamente. Supondremos una aritmética de dígitos finitos provista por

$$\begin{aligned} x \oplus y &= fl(fl(x) + fl(y)), & x \otimes y &= fl(fl(x) \times fl(y)), \\ x \ominus y &= fl(fl(x) - fl(y)), & x \oslash y &= fl(fl(x) \div fl(y)). \end{aligned}$$

Esta aritmética corresponde a realizar aritmética exacta en las representaciones de punto flotante de x y y , después, convertir el resultado exacto a su representación de punto flotante de dígitos finitos.

Ejemplo 3 Suponga que $x = \frac{5}{7}$ y $y = \frac{1}{3}$. Utilice el corte de cinco dígitos para calcular $x + y$, $x - y$, $x \times y$, y $x \div y$.

Solución Observe que

$$x = \frac{5}{7} = 0.\overline{714285} \quad y = \frac{1}{3} = 0.\overline{3}$$

implica que los valores de corte de cinco dígitos de x y y son

$$fl(x) = 0.71428 \times 10^0 \quad y \quad fl(y) = 0.33333 \times 10^0.$$

Por lo tanto,

$$\begin{aligned} x \oplus y &= fl(fl(x) + fl(y)) = fl(0.71428 \times 10^0 + 0.33333 \times 10^0) \\ &= fl(1.04761 \times 10^0) = 0.10476 \times 10^1. \end{aligned}$$

El valor verdadero es $x + y = \frac{5}{7} + \frac{1}{3} = \frac{22}{21}$, por lo que tenemos

$$\text{Error absoluto} = \left| \frac{22}{21} - 0.10476 \times 10^1 \right| = 0.190 \times 10^{-4}$$

y

$$\text{Error relativo} = \left| \frac{0.190 \times 10^{-4}}{22/21} \right| = 0.182 \times 10^{-4}.$$

La tabla 1.2 enumera los valores de éste y otros cálculos. ■

Tabla 1.2

Operación	Resultado	Valor real	Error absoluto	Error relativo
$x \oplus y$	0.10476×10^1	$22/21$	0.190×10^{-4}	0.182×10^{-4}
$x \ominus y$	0.38095×10^0	$8/21$	0.238×10^{-5}	0.625×10^{-5}
$x \otimes y$	0.23809×10^0	$5/21$	0.524×10^{-5}	0.220×10^{-4}
$x \oslash y$	0.21428×10^1	$15/7$	0.571×10^{-4}	0.267×10^{-4}

El error relativo máximo para las operaciones en el ejemplo 3 es 0.267×10^{-4} , por lo que la aritmética produce resultados satisfactorios de cinco dígitos. Éste no es el caso en el siguiente ejemplo.

Ejemplo 4 Suponga que además de $x = \frac{5}{7}$ y $y = \frac{1}{3}$ tenemos

$$u = 0.714251, \quad v = 98765.9, \quad y \quad w = 0.111111 \times 10^{-4},$$

de tal forma que

$$fl(u) = 0.71425 \times 10^0, \quad fl(v) = 0.98765 \times 10^5, \quad y \quad fl(w) = 0.11111 \times 10^{-4}.$$

Determine los valores de corte de cinco dígitos de $x \ominus u$, $(x \ominus u) \oplus w$, $(x \ominus u) \otimes v$, y $u \oplus v$.

Solución Estos números fueron seleccionados para ilustrar algunos problemas que pueden surgir con la aritmética de dígitos finitos. Puesto que x y u son casi iguales, su diferencia es pequeña. El error absoluto para $x \ominus u$ es

$$\begin{aligned} |(x - u) - (x \ominus u)| &= |(x - u) - (fl(fl(x) - fl(u)))| \\ &= \left| \left(\frac{5}{7} - 0.714251 \right) - (fl(0.71428 \times 10^0 - 0.71425 \times 10^0)) \right| \\ &= |0.347143 \times 10^{-4} - fl(0.00003 \times 10^0)| = 0.47143 \times 10^{-5}. \end{aligned}$$

Esta aproximación tiene un error absoluto, pero un error relativo grande

$$\left| \frac{0.47143 \times 10^{-5}}{0.347143 \times 10^{-4}} \right| \leq 0.136.$$

La división subsiguiente entre el número pequeño w o la multiplicación por el número grande v magnifica el error absoluto sin modificar el error relativo. La suma de los números grande y pequeño u y v produce un error absoluto grande, pero no un error relativo grande. Estos cálculos se muestran en la tabla 1.3. ■

Tabla 1.3

Operación	Resultado	Valor real	Error absoluto	Error relativo
$x \ominus u$	0.30000×10^{-4}	0.34714×10^{-4}	0.471×10^{-5}	0.136
$(x \ominus u) \oplus w$	0.27000×10^1	0.31242×10^1	0.424	0.136
$(x \ominus u) \otimes v$	0.29629×10^1	0.34285×10^1	0.465	0.136
$u \oplus v$	0.98765×10^5	0.98766×10^5	0.161×10^1	0.163×10^{-4}

Uno de los cálculos más comunes que producen errores implica la cancelación de dígitos significativos debido a la resta de números casi iguales. Suponga que dos números casi iguales x y y , con $x > y$, tienen las representaciones de dígitos k

$$fl(x) = 0.d_1d_2 \dots d_p\alpha_{p+1}\alpha_{p+2} \dots \alpha_k \times 10^n$$

y

$$fl(y) = 0.d_1d_2 \dots d_p\beta_{p+1}\beta_{p+2} \dots \beta_k \times 10^n.$$

El formato de punto flotante de $x - y$ es

$$fl(fl(x) - fl(y)) = 0.\sigma_{p+1}\sigma_{p+2} \dots \sigma_k \times 10^{n-p},$$

donde

$$0.\sigma_{p+1}\sigma_{p+2} \dots \sigma_k = 0.\alpha_{p+1}\alpha_{p+2} \dots \alpha_k - 0.\beta_{p+1}\beta_{p+2} \dots \beta_k.$$

El número de punto flotante que se usa para representar $x - y$ tiene por lo menos $k - p$ dígitos significativos. Sin embargo, en muchos dispositivos de cálculo a $x - y$ se le asignarán k dígitos, con la última p igual a cero o asignada de manera aleatoria. Cualquier otro cálculo relacionado con $x - y$ conserva el problema de tener solamente $k - p$ dígitos significativos, puesto que una cadena de cálculos no es más precisa que su parte más débil.

Si una representación o un cálculo de dígitos finitos presenta un error, otra ampliación del error ocurre al dividir entre un número de menor magnitud (o, de manera equivalente, al multiplicar por un número de mayor magnitud). Suponga, por ejemplo, que el número z tiene una aproximación de dígitos finitos $z + \delta$, en donde el error δ se introduce por representación o por cálculo previo. Ahora divida entre $\varepsilon = 10^{-n}$, en donde $n > 0$. Entonces

$$\frac{z}{\varepsilon} \approx fl\left(\frac{fl(z)}{fl(\varepsilon)}\right) = (z + \delta) \times 10^n.$$

El error absoluto en esta aproximación, $|\delta| \times 10^n$, es el error absoluto original, $|\delta|$, multiplicado por el factor 10^n .

Ejemplo 5 Si $p = 0.54617$ y $q = 0.54601$. Use aritmética de cuatro dígitos para aproximar $p - q$ y determine los errores absoluto y relativo mediante **a)** redondeo y **b)** corte.

Solución El valor exacto de $r = p - q$ es $r = 0.00016$.

- a)** Suponga que se realiza la resta con aritmética de redondeo de cuatro dígitos. Al redondear p y q a cuatro dígitos obtenemos $p^* = 0.5462$ y $q^* = 0.5460$, respectivamente, y $r^* = p^* - q^* = 0.0002$ es la aproximación de cuatro dígitos para r . Puesto que

$$\frac{|r - r^*|}{|r|} = \frac{|0.00016 - 0.0002|}{|0.00016|} = 0.25,$$

el resultado sólo tiene un dígito significativo, mientras p^* y q^* sean precisos para cuatro y cinco dígitos significativos, respectivamente.

- b) Si se usa el corte para obtener los cuatro dígitos, la aproximación de cuatro dígitos para p , q , y r son $p^* = 0.5461$, $q^* = 0.5460$, y $r^* = p^* - q^* = 0.0001$. Esto nos da

$$\frac{|r - r^*|}{|r|} = \frac{|0.00016 - 0.0001|}{|0.00016|} = 0.375,$$

lo que también resulta en un solo dígito significativo de precisión. ■

A menudo, la pérdida de precisión debido al error de redondeo se puede evitar al reformular los cálculos, como se ilustra en el siguiente ejemplo.

Ilustración

La fórmula cuadrática establece que las raíces de $ax^2 + bx + c = 0$, cuando $a \neq 0$, son

$$x_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a} \quad \text{y} \quad x_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}. \quad (1.1)$$

Considere esta fórmula aplicada a la ecuación $x^2 + 62.10x + 1 = 0$, cuyas raíces son aproximadamente

$$x_1 = -0.01610723 \quad \text{y} \quad x_2 = -62.08390.$$

Las raíces x_1 y x_2 de una ecuación cuadrática general están relacionadas con los coeficientes por el hecho de que

$$x_1 + x_2 = -\frac{b}{a} \quad \text{y} \quad x_1 x_2 = \frac{c}{a}.$$

Éste es un caso especial de las fórmulas de Viète para los coeficientes de los polinomios

Usaremos otra vez la aritmética de redondeo de cuatro dígitos en los cálculos para determinar la raíz. En esta ecuación, b^2 es mucho más grande que $4ac$, por lo que el numerador en el cálculo para x_1 implica la resta de números casi iguales. Ya que

$$\begin{aligned} \sqrt{b^2 - 4ac} &= \sqrt{(62.10)^2 - (4.000)(1.000)(1.000)} \\ &= \sqrt{3856. - 4.000} = \sqrt{3852.} = 62.06, \end{aligned}$$

tenemos

$$fl(x_1) = \frac{-62.10 + 62.06}{2.000} = \frac{-0.04000}{2.000} = -0.02000,$$

una aproximación deficiente a $x_1 = -0.01611$, con un error relativo grande

$$\frac{|-0.01611 + 0.02000|}{|-0.01611|} \approx 2.4 \times 10^{-1}.$$

Por otro lado, el cálculo para x_2 implica la suma de los números casi iguales $-b$ y $-\sqrt{b^2 - 4ac}$. Esto no presenta problemas debido a que

$$fl(x_2) = \frac{-62.10 - 62.06}{2.000} = \frac{-124.2}{2.000} = -62.10$$

tiene un error relativo pequeño

$$\frac{|-62.08 + 62.10|}{|-62.08|} \approx 3.2 \times 10^{-4}.$$

Para obtener una aproximación por redondeo de cuatro dígitos para x_1 , modificamos el formato de la fórmula cuadrática al *racionalizar el numerador*:

$$x_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a} \left(\frac{-b - \sqrt{b^2 - 4ac}}{-b - \sqrt{b^2 - 4ac}} \right) = \frac{b^2 - (b^2 - 4ac)}{2a(-b - \sqrt{b^2 - 4ac})},$$

la cual se simplifica en una fórmula cuadrática alterna:

$$x_1 = \frac{-2c}{b + \sqrt{b^2 - 4ac}}. \quad (1.2)$$

Por medio de la ecuación (1.2) obtenemos

$$fl(x_1) = \frac{-2.000}{62.10 + 62.06} = \frac{-2.000}{124.2} = -0.01610,$$

que tiene el error relativo pequeño 6.2×10^{-4} .

La técnica de racionalización también puede aplicarse para proporcionar la siguiente fórmula cuadrática alterna para x_2 :

$$x_2 = \frac{-2c}{b - \sqrt{b^2 - 4ac}}. \quad (1.3)$$

Éste es el formato que se usa si b es un número negativo. En la ilustración, sin embargo, el uso erróneo de esta fórmula para x_2 no sólo resultaría en la resta de números casi iguales, sino también en la división entre el resultado pequeño de esta resta. La falta de precisión que produce esta combinación,

$$fl(x_2) = \frac{-2c}{b - \sqrt{b^2 - 4ac}} = \frac{-2.000}{62.10 - 62.06} = \frac{-2.000}{0.04000} = -50.00,$$

tiene el error relativo grande 1.9×10^{-1} . ■

- La lección: ¡Piense antes de calcular!

Aritmética anidada

La pérdida de precisión debido a un error de redondeo también se puede reducir al reacomodar los cálculos, como se muestra en el siguiente ejemplo.

Ejemplo 6 Evalúe $f(x) = x^3 - 6.1x^2 + 3.2x + 1.5$ en $x = 4.71$ con aritmética de tres dígitos.

Solución La tabla 1.4 provee los resultados intermedios de los cálculos.

Tabla 1.4

	x	x^2	x^3	$6.1x^2$	$3.2x$
Exacto	4.71	22.1841	104.487111	135.32301	15.072
Tres dígitos (corte)	4.71	22.1	104.	134.	15.0
Tres dígitos (redondeo)	4.71	22.2	105.	135.	15.1

Para ilustrar los cálculos, observemos los que participan para encontrar x^3 usando la aritmética de redondeo de tres dígitos. Primero encontramos

$$x^2 = 4.71^2 = 22.1841 \quad \text{que se redondea a } 22.2.$$

A continuación, usamos este valor de x^2 para encontrar

$$x^3 = x^2 \cdot x = 22.2 \cdot 4.71 = 104.562 \quad \text{que se redondea a } 105.$$

Además,

$$6.1x^2 = 6.1(22.2) = 135.42 \quad \text{que se redondea a } 135,$$

y

$$3.2x = 3.2(4.71) = 15.072 \quad \text{que se redondea a } 15.1.$$

El resultado exacto de la evaluación es

$$\text{Exacto: } f(4.71) = 104.487111 - 135.32301 + 15.072 + 1.5 = -14.263899.$$

Con la aritmética de dígitos finitos, la forma en la que sumamos los resultados puede afectar el resultado final. Suponga que lo hacemos de izquierda a derecha. Entonces, para la aritmética de corte tenemos

$$\text{Tres dígitos (corte): } f(4.71) = ((104. - 134.) + 15.0) + 1.5 = -13.5,$$

y para la aritmética de redondeo tenemos

$$\text{Tres dígitos (redondeo): } f(4.71) = ((105. - 135.) + 15.1) + 1.5 = -13.4.$$

(Usted debe verificar de manera cuidadosa estos resultados para asegurarse de que su noción de aritmética de dígitos finitos es correcta.) Observe que los valores de corte de tres dígitos sólo retienen los tres dígitos principales, sin incluir redondeo, y difieren significativamente de los valores de redondeo de tres dígitos.

Los errores relativos para los métodos de tres dígitos son

$$\text{Corte: } \left| \frac{-14.263899 + 13.5}{-14.263899} \right| \approx 0.05, \quad \text{y redondeo: } \left| \frac{-14.263899 + 13.4}{-14.263899} \right| \approx 0.06.$$

Ilustración

Recuerde que el corte (o redondeo) se realiza después del cálculo.

Como enfoque alternativo, el polinomio $f(x)$ en el ejemplo 6 se puede reescribir de forma **anidada** como

$$f(x) = x^3 - 6.1x^2 + 3.2x + 1.5 = ((x - 6.1)x + 3.2)x + 1.5.$$

Ahora, la aritmética de corte de tres dígitos produce

$$\begin{aligned} f(4.71) &= ((4.71 - 6.1)4.71 + 3.2)4.71 + 1.5 = ((-1.39)(4.71) + 3.2)4.71 + 1.5 \\ &= (-6.54 + 3.2)4.71 + 1.5 = (-3.34)4.71 + 1.5 = -15.7 + 1.5 = -14.2. \end{aligned}$$

De manera similar, ahora obtenemos una respuesta de redondeo de tres dígitos de -14.3 . Los nuevos errores relativos son

$$\begin{aligned} \text{Tres dígitos (corte):} \quad & \left| \frac{-14.263899 + 14.2}{-14.263899} \right| \approx 0.0045; \\ \text{Tres dígitos (redondeo):} \quad & \left| \frac{-14.263899 + 14.3}{-14.263899} \right| \approx 0.0025. \end{aligned}$$

El anidado ha reducido el error relativo para la aproximación de corte a menos de 10% del valor obtenido al inicio. Para la aproximación de redondeo, la mejora ha sido todavía más drástica; el error, en este caso, se ha reducido más de 95 por ciento.

Los polinomios *siempre* deberían expresarse en forma anidada antes de realizar una evaluación porque esta forma minimiza el número de cálculos aritméticos. La disminución del error en la ilustración se debe a la reducción de los cálculos de cuatro multiplicaciones y tres sumas a dos multiplicaciones y tres sumas. Una forma de disminuir el error de redondeo es reducir el número de cálculos.

La sección Conjunto de ejercicios 1.2 está disponible en línea. Encuentre la ruta de acceso en las páginas preliminares.

1.3 Algoritmos y convergencia

El uso de algoritmos es tan antiguo como las matemáticas formales pero el nombre se deriva del matemático árabe Muhammad ibn-Msâ al-Khwarîzmî (c. 780–850). La traducción latina de sus trabajos comenzó con las palabras “Dixit Algorismi”, que significan “al-Khwarîzmî dice”.

A lo largo del texto examinaremos procedimientos de aproximación, llamados *algoritmos*, los cuales incluyen secuencias de cálculos. Un **algoritmo** es un procedimiento que describe, de manera inequívoca, una secuencia finita de pasos que se desarrollarán en un orden específico. El objeto del algoritmo es implementar un procedimiento para resolver un problema o aproximar una solución para el problema.

Nosotros usamos un **pseudocódigo** para describir los algoritmos. Este pseudocódigo describe la forma de la entrada que se proporcionará y la forma de la salida deseada. No todos los procedimientos proporcionan una salida satisfactoria para una entrada seleccionada de manera arbitraria. Como consecuencia, se incluye una técnica de detención independiente de la técnica numérica en cada algoritmo para evitar ciclos infinitos.

En los algoritmos se usan dos símbolos de puntuación:

- Un punto (.) indica el final de un paso.
- Punto y coma (;) separan las tareas dentro de un paso.

La sangría se usa para indicar que los grupos de declaraciones se tratarán como una sola entidad.

Las técnicas de ciclado en los algoritmos también son controladas por contador, como

Para	$i = 1, 2, \dots, n$
tome	$x_i = a + i \cdot h$

O controladas por condición, como

Mientras $i < N$ realice los pasos 3–6.

Para permitir una ejecución condicional, usamos las construcciones estándar

Si... entonces	o	Si... entonces
		si no

Los pasos en los algoritmos siguen las reglas de la construcción del programa estructurado. Se han ordenado de tal forma que no debería ser difícil traducir el pseudocódigo en cualquier lenguaje de programación adecuado para aplicaciones científicas.

Los algoritmos están mezclados libremente con comentarios. Éstos se escriben en itálicas y se encuentran entre paréntesis para distinguirlos de las declaraciones algorítmicas.

NOTA: Cuando es difícil determinar el final de ciertos pasos anidados utilizamos un comentario como (fin del paso 14) a la derecha o debajo de la declaración de finalización. Consulte, por ejemplo, el comentario en el paso 5, en el ejemplo 1.

Ilustración

El siguiente algoritmo calcula $x_1 + x_2 + \dots + x_N = \sum_{i=1}^N x_i$, dado N y los números $x_1, x_2, \dots, x_N$

ENTRADA $N, x_1, x_2, \dots, x_n$.

SALIDA $SUM = \sum_{i=1}^N x_i$.

Paso 1 Tome $SUM = 0$. (*Inicialice el acumulador.*)

Paso 2 Para $i = 1, 2, \dots, N$ hacer
Tome $SUM = SUM + x_i$. (*Añadir el siguiente término.*)

Paso 3 SALIDA (SUM);
PARE.

Ejemplo 1 El n -ésimo polinomio de Taylor para $f(x) = \ln x$ ampliado alrededor de $x_0 = 1$ es

$$P_N(x) = \sum_{i=1}^N \frac{(-1)^{i+1}}{i} (x-1)^i,$$

y el valor de $\ln 1.5$ para los ocho lugares decimales es 0.40546511. Construya un algoritmo para determinar el valor mínimo de N requerido para

$$|\ln 1.5 - P_N(1.5)| < 10^{-5}$$

sin utilizar el término restante del polinomio de Taylor.

Solución A partir del cálculo sabemos que si $\sum_{n=1}^{\infty} a_n$ es una serie alterna con límite A cuyos términos disminuyen en magnitud, entonces A y la n -ésima suma parcial $A_N = \sum_{n=1}^N a_n$ difieren por menos en la magnitud del término $(N+1)$; es decir,

$$|A - A_N| \leq |a_{N+1}|.$$

El siguiente algoritmo utiliza este hecho.

ENTRADA valor x , tolerancia TOL , número máximo de iteraciones M .

SALIDA grado N del polinomio o un mensaje de falla.

Paso 1 Sea $N = 1$;

$y = x - 1$;

$SUM = 0$;

$POWER = y$;

$TERM = y$;

$SIGN = -1$. (Se utiliza para implementar la alternancia de los signos.)

Paso 2 Mientras $N \leq M$ haga los pasos 3–5.

Paso 3 Determine $SIGN = -SIGN$; (Alterne los signos.)

$SUM = SUM + SIGN \cdot TERM$; (Acumule los términos.)

$POWER = POWER \cdot y$;

$TERM = POWER/(N+1)$. (Calcule el siguiente término.)

Paso 4 Si $|TERM| < TOL$ entonces (Prueba para la precisión.)

SALIDA (N);

PARE. (El procedimiento fue exitoso.)

Paso 5 Determinar $N = N + 1$. (Preparar la siguiente iteración. (Fin del paso 2))

Paso 6 **SALIDA** ('El método falló'); (El procedimiento no fue exitoso.)

PARE.

La entrada para nuestro problema es $x = 1.5$, $TOL = 10^{-5}$ y tal vez $M = 15$. Esta elección de M provee un límite o una cota superior para el número de cálculos que queremos realizar, al reconocer que el algoritmo probablemente va a fallar si se excede este límite. La salida es un valor para N o el mensaje de fracaso que depende de la precisión del dispositivo computacional. ■

Algoritmos de caracterización

Consideraremos diversos problemas de aproximación a lo largo del texto y en cada caso necesitamos determinar métodos de aproximación que producen resultados precisos fiables para una amplia clase de problemas. Debido a las diferentes formas de derivar los métodos de aproximación, requerimos una variedad de condiciones para clasificar su precisión. No todas estas condiciones son apropiadas para cualquier problema en particular.

La palabra *estable* tiene la misma raíz que las palabras *posición* y *estándar*. En matemáticas, el término *estable* aplicado a un problema indica que un pequeño cambio en los datos o las condiciones iniciales no resultan en un cambio drástico en la solución del problema.

Un criterio que impondremos en un algoritmo, siempre que sea posible, es que los pequeños cambios en los datos iniciales producen, de forma proporcional, pequeños cambios en los resultados finales. Un algoritmo que satisface esta propiedad recibe el nombre de **estable**; de lo contrario, es **inestable**. Algunos algoritmos son estables sólo para ciertas elecciones de datos iniciales y reciben el nombre de **estables condicionalmente**. Clasificaremos las propiedades de estabilidad de los algoritmos siempre que sea posible.

Para considerar más el tema del crecimiento del error de redondeo y su conexión con la estabilidad del algoritmo, suponga que se presenta un error con una magnitud $E_0 > 0$ en alguna etapa en los cálculos y que la magnitud del error después de n operaciones subsiguientes se denota con E_n . En la práctica, los dos casos que surgen con mayor frecuencia se definen a continuación.

Definición 1.17 Suponga que $E_0 > 0$ denota un error que se presenta en alguna etapa en los cálculos y E_n representa la magnitud del error después de n operaciones subsiguientes.

- Si $E_n \approx C^n E_0$, donde C es una constante independiente de n , entonces se dice que el crecimiento del error es **lineal**.
- Si $E_n \approx C^n E_0$, para algunas $C > 1$, entonces el crecimiento del error recibe el nombre de **exponencial**. ■

Normalmente, el crecimiento lineal del error es inevitable, y cuando C y E_0 son pequeñas, en general, los resultados son aceptables. El crecimiento exponencial del error debería evitarse porque el término C^n se vuelve grande incluso para los valores relativamente pequeños de n . Esto conduce a imprecisiones inaceptables, independientemente del tamaño de E_0 . Como consecuencia, mientras un algoritmo que presenta crecimiento lineal del error es estable, un algoritmo que presenta crecimiento exponencial del error es inestable. (Consulte la figura 1.10.)

Figura 1.10

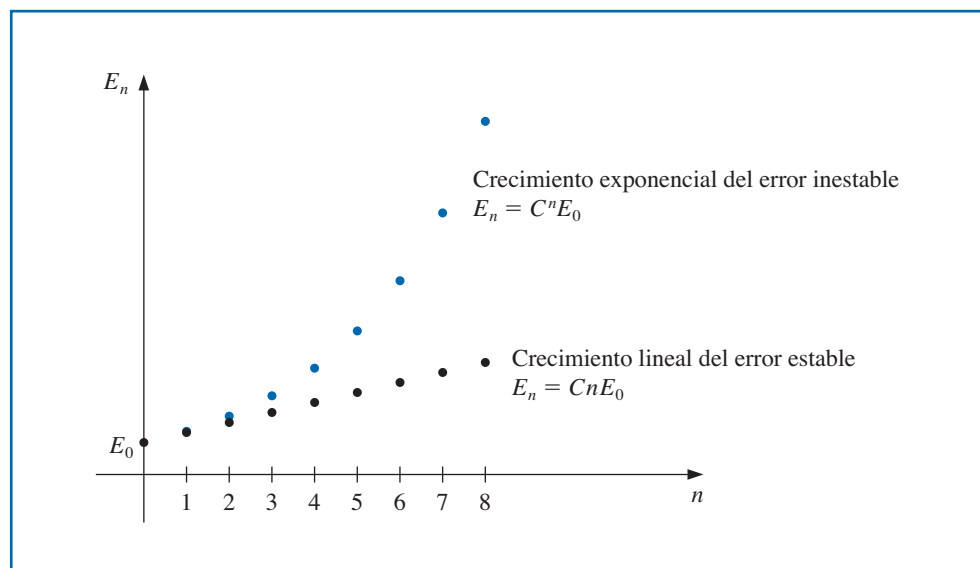


Ilustración Para cualquier constante c_1 y c_2 ,

$$p_n = c_1 \left(\frac{1}{3}\right)^n + c_2 3^n, \quad (1.4)$$

es una solución a la ecuación recursiva

$$p_n = \frac{10}{3} p_{n-1} - p_{n-2}, \quad \text{para } n = 2, 3, \dots$$

Se puede observar que

$$\begin{aligned} \frac{10}{3} p_{n-1} - p_{n-2} &= \frac{10}{3} \left[c_1 \left(\frac{1}{3} \right)^{n-1} + c_2 3^{n-1} \right] - \left[c_1 \left(\frac{1}{3} \right)^{n-2} + c_2 3^{n-2} \right] \\ &= c_1 \left(\frac{1}{3} \right)^{n-2} \left[\frac{10}{3} \cdot \frac{1}{3} - 1 \right] + c_2 3^{n-2} \left[\frac{10}{3} \cdot 3 - 1 \right] \\ &= c_1 \left(\frac{1}{3} \right)^{n-2} \left(\frac{1}{9} \right) + c_2 3^{n-2} (9) = c_1 \left(\frac{1}{3} \right)^n + c_2 3^n = p_n. \end{aligned}$$

Suponga que tenemos $p_0 = 1$ y $p_1 = \frac{1}{3}$. Usando estos valores y la ecuación (1.4) podemos determinar valores únicos para las constantes $c_1 = 1$ y $c_2 = 0$. Por lo tanto, $p_n = \left(\frac{1}{3} \right)^n$ para todas las n .

Si se utiliza aritmética de redondeo de cinco dígitos para calcular los términos de la sucesión determinada por esta ecuación, entonces $\hat{p}_0 = 1.0000$ y $\hat{p}_1 = 0.33333$, lo cual requiere la modificación de las constantes para $\hat{c}_1 = 1.0000$ y $\hat{c}_2 = -0.12500 \times 10^{-5}$. Así, la sucesión generada $\{\hat{p}_n\}_{n=0}^{\infty}$ está dada por

$$\hat{p}_n = 1.0000 \left(\frac{1}{3} \right)^n - 0.12500 \times 10^{-5} (3)^n,$$

que tiene un error de redondeo,

$$p_n - \hat{p}_n = 0.12500 \times 10^{-5} (3^n).$$

Este procedimiento es inestable ya que el error aumenta *exponencialmente* con n , lo cual se refleja en las imprecisiones extremas después de los primeros términos, como se muestra en la tabla 1.5.

Tabla 1.5

n	$\hat{p}_n$ calculada	p_n corregida	Error relativo
0	0.10000×10^1	0.10000×10^1	
1	0.33333×10^0	0.33333×10^0	
2	0.11110×10^0	0.11111×10^0	9×10^{-5}
3	0.37000×10^{-1}	0.37037×10^{-1}	1×10^{-3}
4	0.12230×10^{-1}	0.12346×10^{-1}	9×10^{-3}
5	0.37660×10^{-2}	0.41152×10^{-2}	8×10^{-2}
6	0.32300×10^{-3}	0.13717×10^{-2}	8×10^{-1}
7	-0.26893×10^{-2}	0.45725×10^{-3}	7×10^0
8	-0.92872×10^{-2}	0.15242×10^{-3}	6×10^1

Ahora considere esta ecuación recursiva:

$$p_n = 2p_{n-1} - p_{n-2}, \quad \text{para } n = 2, 3, \dots$$

Tiene la solución $p_n = c_1 + c_2 n$ para cualquier constante c_1 y c_2 porque

$$\begin{aligned} 2p_{n-1} - p_{n-2} &= 2(c_1 + c_2(n-1)) - (c_1 + c_2(n-2)) \\ &= c_1(2-1) + c_2(2n-2-n+2) = c_1 + c_2 n = p_n. \end{aligned}$$

Si tenemos $p_0 = 1$ y $p_1 = \frac{1}{3}$, entonces las constantes en esta ecuación se determinan exclusivamente como $c_1 = 1$ y $c_2 = -\frac{2}{3}$. Esto implica que $p_n = 1 - \frac{2}{3}n$.

Si se utiliza aritmética de redondeo de cinco dígitos para calcular los términos de la sucesión provista por esta ecuación, entonces $\hat{p}_0 = 1.0000$ y $\hat{p}_1 = 0.33333$. Como consecuencia, las constantes de redondeo de cinco dígitos son $\hat{c}_1 = 1.0000$ y $\hat{c}_2 = -0.66667$. Por lo tanto,

$$\hat{p}_n = 1.0000 - 0.66667n,$$

que tiene un error de redondeo

$$p_n - \hat{p}_n = \left(0.66667 - \frac{2}{3}\right)n.$$

Este procedimiento es estable porque el error aumenta *linealmente* con n , lo cual se refleja en las aproximaciones que se muestran en la tabla 1.6. ■

Tabla 1.6

n	$\hat{p}_n$ calculada	p_n corregida	Error relativo
0	0.10000×10^1	0.10000×10^1	
1	0.33333×10^0	0.33333×10^0	
2	-0.33330×10^0	-0.33333×10^0	9×10^{-5}
3	-0.10000×10^1	-0.10000×10^1	0
4	-0.16667×10^1	-0.16667×10^1	0
5	-0.23334×10^1	-0.23333×10^1	4×10^{-5}
6	-0.30000×10^1	-0.30000×10^1	0
7	-0.36667×10^1	-0.36667×10^1	0
8	-0.43334×10^1	-0.43333×10^1	2×10^{-5}

Los efectos del error de redondeo se pueden reducir con la aritmética de dígitos de orden superior, como la opción de precisión doble o múltiple disponible en muchas computadoras. Las desventajas de utilizar la aritmética de precisión doble son que requiere más tiempo de cálculo y el crecimiento del error de redondeo no se elimina por completo.

Un enfoque para calcular el error de redondeo es usar la aritmética de intervalo (es decir, retener los valores más grande y más pequeño posibles), de esta forma, al final, obtenemos un intervalo que contiene el valor verdadero. Por desgracia, podría ser necesario un intervalo pequeño para la implementación razonable.

Tasas de convergencia

Puesto que con frecuencia se utilizan técnicas iterativas relacionadas con sucesiones, esta sección concluye con un análisis breve sobre la terminología que se usa para describir la rapidez con que ocurre la convergencia. En general, nos gustaría que la técnica converja tan rápido como sea posible. La siguiente definición se usa para comparar las tasas de convergencia de las sucesiones.

Definición 1.18 Suponga que $\{\beta_n\}_{n=1}^{\infty}$ es una sucesión conocida que converge a cero y $\{\alpha_n\}_{n=1}^{\infty}$ converge a un número α . Si existe una constante positiva K con

$$|\alpha_n - \alpha| \leq K|\beta_n|, \quad \text{para una } n \text{ grande,}$$

entonces decimos que $\{\alpha_n\}_{n=1}^{\infty}$ converge a α con **una rapidez, u orden de convergencia** $O(\beta_n)$. (Esta expresión se lee “O de β_n ”). Se indica al escribir $\alpha_n = \alpha + O(\beta_n)$. ■

A pesar de que la definición 1.18 permite comparar $\{\alpha_n\}_{n=1}^{\infty}$ con una sucesión arbitraria $\{\beta_n\}_{n=1}^{\infty}$, en casi todas las situaciones usamos

$$\beta_n = \frac{1}{n^p},$$

para algún número $p > 0$. En general, nos interesa el valor más grande de p con $\alpha_n = \alpha + O(1/n^p)$.

Ejemplo 2 Suponga que, para $n \geq 1$,

$$\alpha_n = \frac{n+1}{n^2} \quad \text{y} \quad \hat{\alpha}_n = \frac{n+3}{n^3}.$$

aunque $\lim_{n \rightarrow \infty} \alpha_n = 0$ y $\lim_{n \rightarrow \infty} \hat{\alpha}_n = 0$, la sucesión $\{\hat{\alpha}_n\}$ converge a este límite mucho más rápido que la sucesión $\{\alpha_n\}$. Al usar la aritmética de redondeo de cinco dígitos, tenemos los valores que se muestran en la tabla 1.7. Determine la rapidez de convergencia para estas dos sucesiones.

Tabla 1.7

n	1	2	3	4	5	6	7
α_n	2.00000	0.75000	0.44444	0.31250	0.24000	0.19444	0.16327
$\hat{\alpha}_n$	4.00000	0.62500	0.22222	0.10938	0.064000	0.041667	0.029155

Existen muchas otras formas de describir el crecimiento de las sucesiones y las funciones, algunas requieren límites tanto por encima como por debajo de la sucesión o función que se considera. Cualquier buen libro que analiza algoritmos, por ejemplo, [CLRS], incluiría esta información.

Solución Defina las sucesiones $\beta_n = 1/n$ y $\hat{\beta}_n = 1/n^2$. Entonces

$$|\alpha_n - 0| = \frac{n+1}{n^2} \leq \frac{n+n}{n^2} = 2 \cdot \frac{1}{n} = 2\beta_n$$

y

$$|\hat{\alpha}_n - 0| = \frac{n+3}{n^3} \leq \frac{n+3n}{n^3} = 4 \cdot \frac{1}{n^2} = 4\hat{\beta}_n.$$

Por lo tanto, la rapidez de convergencia de $\{\alpha_n\}$ a cero es similar a la convergencia de $\{1/n\}$ a cero, mientras $\{\hat{\alpha}_n\}$ converge a cero con una rapidez similar para la sucesión que converge más rápido $\{1/n^2\}$. Expresamos esto al escribir

$$\alpha_n = 0 + O\left(\frac{1}{n}\right) \quad \text{y} \quad \hat{\alpha}_n = 0 + O\left(\frac{1}{n^2}\right). \quad \blacksquare$$

También usamos la notación O (*O grande*) para describir la rapidez con la que convergen las funciones.

Definición 1.19 Suponga que $\lim_{h \rightarrow 0} G(h) = 0$ y $\lim_{h \rightarrow 0} F(h) = L$. Si existe una constante positiva K con

$$|F(h) - L| \leq K|G(h)|, \quad \text{para } h \text{ suficientemente pequeña,}$$

entonces escribimos $F(h) = L + O(G(h))$. $\blacksquare$

En general, las funciones que utilizamos para comparar tienen la forma de $G(h) = h^p$, donde $p > 0$. Nos interesa el valor más grande de p , para el que $F(h) = L + O(h^p)$.

Ejemplo 3 Use el tercer polinomio de Taylor alrededor de $h = 0$ para mostrar que $\cos h + \frac{1}{2}h^2 = 1 + O(h^4)$.

Solución En el ejemplo 3b) de la sección 1.1, vimos que este polinomio es

$$\cos h = 1 - \frac{1}{2}h^2 + \frac{1}{24}h^4 \cos \tilde{\xi}(h),$$

para algún número $\tilde{\xi}(h)$ entre cero y h . Esto implica que

$$\cos h + \frac{1}{2}h^2 = 1 + \frac{1}{24}h^4 \cos \tilde{\xi}(h).$$

Por lo tanto,

$$\left| \left(\cos h + \frac{1}{2}h^2 \right) - 1 \right| = \left| \frac{1}{24} \cos \tilde{\xi}(h) \right| h^4 \leq \frac{1}{24}h^4,$$

de modo que $h \rightarrow 0$, $\cos h + \frac{1}{2}h^2$ converge a este límite, 1, tan rápido como h^4 converge a 0. Es decir,

$$\cos h + \frac{1}{2}h^2 = 1 + O(h^4). \quad \blacksquare$$

La sección Conjunto de ejercicios 1.3 está disponible en línea. Encuentre la ruta de acceso en las páginas preliminares.

1.4 Software numérico

Los paquetes de software computacional para aproximar las soluciones numéricas a los problemas en muchas formas. En nuestro sitio web para el libro

<https://sites.google.com/site/numericalanalysis1burden/>

proporcionamos programas escritos en C, FORTRAN, Maple, Mathematica, MATLAB y Pascal, así como applets de JAVA. Éstos se pueden utilizar para resolver los problemas provistos en los ejemplos y ejercicios, y aportan resultados satisfactorios para muchos de los problemas que usted quizá necesite resolver. Sin embargo, son lo que llamamos programas de *propósito especial*. Nosotros usamos este término para distinguir estos programas de aquellos disponibles en las librerías de subrutinas matemáticas estándar. Los programas en estos paquetes recibirán el nombre de *propósito general*.

Los programas en los paquetes de software de propósito general difieren en sus intenciones de los algoritmos y programas proporcionados en este libro. Los paquetes de software de propósito general consideran formas para reducir los errores debido al redondeo de máquina, el subdesbordamiento y el desbordamiento. También describen el rango de entrada que conducirá a los resultados con cierta precisión específica. Éstas son características dependientes de la máquina, por lo que los paquetes de software de propósito general utilizan parámetros que describen las características de punto flotante de la máquina que se usa para los cálculos.

Ilustración Para ilustrar algunas diferencias entre los programas incluidos en un paquete de propósito general y un programa que nosotros proporcionaríamos en este libro, consideremos un algoritmo que calcula la norma euclidiana de un vector n dimensional $\mathbf{x} = (x_1, x_2, \dots, x_n)^t$. A menudo, esta norma se requiere dentro de los programas más grandes y se define mediante

$$\|\mathbf{x}\|_2 = \left[\sum_{i=1}^n x_i^2 \right]^{1/2}.$$

La norma da una medida de la distancia del vector $\mathbf{x}$ y el vector $\mathbf{0}$. Por ejemplo, el vector $\mathbf{x} = (2, 1, 3, 2, 1)^t$ tiene

$$\|\mathbf{x}\|_2 = [2^2 + 1^2 + 3^2 + (-2)^2 + (-1)^2]^{1/2} = \sqrt{19},$$

por lo que su distancia a partir de $\mathbf{0} = (0, 0, 0, 0, 0)^t$ es $\sqrt{19} \approx 4.36$.

Un algoritmo del tipo que presentaríamos para este problema se proporciona aquí. No incluye parámetros dependientes de máquina y no ofrece garantías de precisión, pero aportará resultados precisos “la mayor parte del tiempo”.

ENTRADA $n, x_1, x_2, \dots, x_n$.

SALIDA $NORM$.

Paso 1 Haga $SUM = 0$.

Paso 2 Para $i = 1, 2, \dots, n$ determine $SUM = SUM + x_i^2$.

Paso 3 Determine $NORM = SUM^{1/2}$.

Paso 4 SALIDA ($NORM$);
PARE.

Un programa con base en nuestro algoritmo es fácil de escribir y comprender. Sin embargo, el programa no sería suficientemente preciso debido a diferentes razones. Por ejemplo, la magnitud de algunos números podría ser demasiado grande o pequeña para representarse con precisión en el sistema de punto flotante de la computadora. Además, este orden para realizar los cálculos quizá no produzca los resultados más precisos, o que la rutina para obtener la raíz cuadrada podría no ser la mejor disponible para el problema. Los diseñadores del algoritmo consideran asuntos de este tipo al escribir programas para software de propósito general. A menudo, estos programas contienen subprogramas para resolver problemas más amplios, por lo que deben incluir controles que nosotros no necesitaremos.

Algoritmos de propósito general

Ahora consideremos un algoritmo para un programa de software de propósito general para calcular la norma euclidiana. Primero, es posible que a pesar de que un componente x_i del vector se encuentre dentro del rango de la máquina, el cuadrado del componente no lo esté. Esto se puede presentar cuando alguna $|x_i|$ es tan pequeña que x_i^2 causa subdesbordamiento o cuando alguna $|x_i|$ es tan grande que x_i^2 causa desbordamiento.

También es posible que todos estos términos se encuentren dentro del rango de la máquina, pero que ocurra desbordamiento a partir de la suma de un cuadrado de uno de los términos para la suma calculada previamente.

Los criterios de precisión dependen de la máquina en la que se realizan los cálculos, por lo que los parámetros dependientes de la máquina se incorporan en el algoritmo. Suponga que trabajamos en una computadora hipotética con base 10, la cual tiene $t \geq 4$ dígitos de precisión, un exponente mínimo $emín$ y un exponente máximo $emáx$. Entonces el conjunto de números de punto flotante en esta máquina consiste en 0 y los números de la forma

$$x = f \cdot 10^e, \quad \text{donde} \quad f = \pm(f_1 10^{-1} + f_2 10^{-2} + \dots + f_t 10^{-t}),$$

donde $1 \leq f_1 \leq 9$ y $0 \leq f_i \leq 9$, para cada $i = 2, \dots, t$, y donde $emín \leq e \leq emáx$. Estas restricciones implican que el número positivo más pequeño representado en la máquina es $\sigma = 10^{emín-1}$, por lo que cualquier número calculado x con $|x| < \sigma$ causa subdesbordamiento y que x sea 0. El número positivo más grande es $\lambda = (1 - 10^{-t})10^{emáx}$, y cualquier número calculado x con $|x| > \lambda$ causa desbordamiento. Cuando se presenta subdesbordamiento, el programa continuará, a menudo, sin una pérdida significativa de precisión. Si se presenta desbordamiento, el programa fallará.

El algoritmo supone que las características de punto flotante de la máquina se describen a través de los parámetros N , s , S , y y Y . El número máximo de entradas que se pueden sumar con por lo menos $t/2$ dígitos de precisión está provisto por N . Esto implica que el algoritmo procederá a encontrar la norma de un vector $x = (x_1, x_2, \dots, x_n)^t$ sólo si $n \leq N$. Para resolver el problema de subdesbordamiento-desbordamiento, los números de punto flotante distintos a cero se dividen en tres grupos:

- números de magnitud pequeña x , aquellos que satisfacen $0 < |x| < y$;
- números de magnitud media x , donde $y \leq |x| < Y$;
- números de magnitud grande x , donde $Y \leq |x|$.

Los parámetros y y Y se seleccionan con el fin de evitar el problema de subdesbordamiento-desbordamiento al sumar y elevar al cuadrado los números de magnitud media. Elevar al cuadrado los números de magnitud pequeña puede causar subdesbordamiento, por lo que se utiliza un factor de escala S mucho mayor a 1 con el resultado $(sx)^2$ que evita el subdesbordamiento incluso cuando x^2 no lo hace. Sumar y elevar al cuadrado los números que tienen una magnitud grande puede causar desbordamiento. Por lo que, en este caso, se utiliza un factor de escala positivo s mucho menor a 1 para garantizar que $(sx)^2$ no cause desbordamiento al calcularlo o incluirlo en una suma, a pesar de que x^2 lo haría.

Para evitar escalamiento innecesario, y y Y se seleccionan de tal forma que el rango de números de magnitud media sea tan largo como sea posible. El siguiente algoritmo es una modificación del que se describe en [Brow, W], p. 471. Éste incluye un procedimiento para sumar los componentes escalados del vector, que son de magnitud pequeña hasta encontrar un componente de magnitud media. Entonces se elimina la escala de la suma previa y continúa al elevar al cuadrado y sumar los números pequeños y medianos hasta encontrar un componente con una magnitud grande. Una vez que el componente con magnitud grande aparece, el algoritmo escala la suma anterior y procede a escalar, elevar al cuadrado y sumar los números restantes.

El algoritmo supone que, en la transición desde números pequeños a medianos, los números pequeños no escalados son despreciables, al compararlos con números medianos. De igual forma, en la transición desde números medianos a grandes, los números medianos no escalados son despreciables, al compararlos con números grandes. Por lo tanto, las selecciones de los parámetros de escalamiento se deben realizar de tal forma que se igualen a 0 sólo cuando son verdaderamente despreciables. Las relaciones comunes entre las características de máquina, como se describen en t , σ , λ , $emín$ y $emáx$ y los parámetros del algoritmo N , s , S , y y Y se determinan después del algoritmo.

El algoritmo usa tres indicadores para señalar las diferentes etapas en el proceso de suma. Estos indicadores son valores iniciales determinados en el paso 3 del algoritmo. FLAG (BANDERA) 1 es 1 hasta encontrar un componente mediano o grande; entonces se convierte en 0. FLAG (BANDERA) 2 es 0 mientras se suman números pequeños, cambia a 1 cuando se encuentra un número mediano por primera vez, y regresa a 0 cuando se encuentra un número grande. Inicialmente, FLAG (BANDERA) 3 es 0 y cambia a 1 cuando se encuentra un número grande por primera vez. El paso 3 también introduce el indicador DONE (HECHO), que es 0 hasta que se terminan los cálculos y, entonces, regresa a 1.

ENTRADA $N, s, S, y, Y, \lambda, n, x_1, x_2, \dots, x_n$.

SALIDA $NORM$ o un mensaje de error apropiado.

Paso 1 Si $n \leq 0$ entonces SALIDA ('El entero n debe ser positivo.')

PARE.

Paso 2 Si $n \geq N$ entonces SALIDA ('El entero n es demasiado grande.')

PARE.

Paso 3 Determine $SUM = 0$;
 $FLAG1 = 1$; (Se suman los números pequeños.)
 $FLAG2 = 0$;
 $FLAG3 = 0$;
 $DONE = 0$;
 $i = 1$.

Paso 4 Mientras ($i \leq n$ y $FLAG1 = 1$) haga el paso 5.

Paso 5 Si $|x_i| < y$ entonces determine $SUM = SUM + (Sx_i)^2$;
 $i = i + 1$
también determine $FLAG1 = 0$. (Se encuentra un número no pequeño.)

Paso 6 Si $i > n$ entonces determine $NORM = (SUM)^{1/2}/S$;
 $DONE = 1$
también determine $SUM = (SUM/S)/S$; (Escalamiento de números grandes.)
 $FLAG2 = 1$.

Paso 7 Mientras ($i \leq n$ y $FLAG2 = 1$) haga el paso 8. (Se suman los números medianos.)

Paso 8 Si $|x_i| < Y$ entonces determine $SUM = SUM + x_i^2$;
 $i = i + 1$
también determine $FLAG2 = 0$. (Se encuentra un número no grande.)

Paso 9 Si $DONE = 0$ entonces
si $i > n$ entonces determine $NORM = (SUM)^{1/2}$;
 $DONE = 1$
también determine $SUM = ((SUM)s)s$; (Escalamiento de números grandes.)
 $FLAG3 = 1$.

Paso 10 Mientras ($i \leq n$ y $FLAG3 = 1$) haga el paso 11.

Paso 11 Determine $SUM = SUM + (sx_i)^2$; (Sume los números grandes.)
 $i = i + 1$.

Paso 12 Si $DONE = 0$ entonces
si $SUM^{1/2} < \lambda s$ entonces determine $NORM = (SUM)^{1/2}/s$;
 $DONE = 1$
también determine $SUM = \lambda$. (La norma es demasiado grande.)

Paso 13 Si $DONE = 1$ entonces SALIDA ('Norma es', $NORM$)
también SALIDA ('Norma $\geq$ ', $NORM$, 'ocurrió sobreflujo').

Paso 14 PARE.

Las relaciones entre las características de máquina $t, \sigma, \lambda, emín$ y $emáx$ y los parámetros del algoritmo N, s, S, y y Y se seleccionaron en [Brow, W], p. 471, como

$N = 10^{e_N}$, donde $e_N = \lfloor (t - 2)/2 \rfloor$, El entero más grande menor o igual a $(t - 2)/2$;

$s = 10^{e_s}$, donde $e_s = \lfloor -(emáx + e_N)/2 \rfloor$;

$S = 10^{e_S}$, donde $e_S = \lceil (1 - emín)/2 \rceil$, El entero más pequeño mayor o igual que $(1 - emín)/2$;

$y = 10^{e_y}$, donde $e_y = \lceil (emín + t - 2)/2 \rceil$;

$Y = 10^{e_Y}$, donde $e_Y = \lfloor (emáx - e_N)/2 \rfloor$.

La primera computadora portátil fue la Osborne I, producida en 1981, a pesar de que era mucho más grande y pesada de lo que podríamos pensar como portátil.

El sistema FORTRAN (FORMula TRANslator) fue el lenguaje de programación científica de propósito general original. Sigue utilizándose ampliamente en situaciones que requieren cálculos científicos intensivos.

El proyecto EISPACK fue el primer paquete de software numérico a gran escala en estar disponible para dominio público y lideró el camino para que muchos paquetes lo siguieran.

La ingeniería de software se estableció como disciplina de laboratorio durante las décadas de 1970 y 1980. EISPACK se desarrolló en Argonne Labs y LINPACK poco después. A principios de la década de 1980, Argonne fue reconocido en el ámbito internacional como líder mundial en cálculos simbólicos y numéricos.

En 1970, IMSL se convirtió en la primera librería científica a gran escala para computadoras centrales. Ya que en esa época, existían librerías para sistemas computacionales que iban desde supercomputadoras hasta computadoras personales.

La fiabilidad construida en este algoritmo ha incrementado ampliamente la complejidad, en comparación con el algoritmo provisto antes en esta sección. En la mayoría de los casos, los algoritmos de propósito especial y general proporcionan resultados idénticos. La ventaja del algoritmo de propósito general es que proporciona seguridad para sus resultados.

Existen muchas formas de software numérico de propósito general disponibles en el ámbito comercial y en el dominio público. La mayor parte de los primeros se escribió para las computadoras centrales, y una buena referencia es *Sources and Development of Mathematical Software (Fuentes y desarrollo de software matemático)*, editado por Wayne Cowell [Co].

Ahora que las computadoras personales son suficientemente poderosas, existe software numérico estándar para ellas. La mayoría de este software numérico se escribe en FORTRAN, a pesar de que algunos están escritos en C, C++ y FORTRAN90.

Los procedimientos ALGOL se presentaron para cálculos de matrices en 1971 en [WR]. Después, se desarrolló un paquete de subrutinas FORTRAN con base principalmente en los procedimientos ALGOL dentro de las rutinas EISPACK. Estas rutinas se documentan en los manuales publicados por Springer-Verlag como parte de sus *Lecture Notes (Notas de clase)* en la serie *Computer Science (Ciencias computacionales)* [Sm, B] y [Gar]. Las subrutinas FORTRAN se utilizan para calcular los valores propios y los vectores propios para una variedad de diferentes tipos de matrices.

LINPACK es un paquete de subrutinas FORTRAN para analizar y resolver sistemas de ecuaciones lineales y resolver problemas de mínimos cuadrados lineales. La documentación en este paquete se encuentra en [DBMS]. Una introducción paso a paso para LINPACK, EISPACK y BLAS (Basic Linear Algebra Subprograms; Subprogramas de Álgebra Lineal Básica) se proporciona en [CV].

El paquete LAPACK, disponible por primera vez en 1992, es una librería de las subrutinas FORTRAN que sustituyen a LINPACK y EISPACK al integrar estos dos conjuntos de algoritmos en un paquete unificado y actualizado. El software se ha reestructurado para lograr mayor eficiencia en procesadores de vectores y otros multiprocesadores de alto desempeño y memoria compartida. LAPACK se expande en profundidad y amplitud en la versión 3.0, disponible en FORTRAN, FORTRAN90, C, C++ y JAVA. C y JAVA sólo están disponibles como interfaces de idioma o traducciones de las librerías FORTRAN de LAPACK. El paquete BLAS no forma parte de LAPACK, pero el código para BLAS se distribuye con LAPACK.

Otros paquetes para resolver tipos específicos de problemas están disponibles en el dominio público. Como alternativa para netlib, puede utilizar Xnetlib para buscar en la base de datos y recuperar software. Encuentre más información en el artículo *Software Distribution Using Netlib* de Dongarra Roman y Wade [DRW].

Estos paquetes de software son muy eficientes, precisos y confiables. Se prueban de manera meticulosa y la documentación está disponible fácilmente. A pesar de que los paquetes son portátiles, es una buena idea investigar la dependencia de la máquina y leer la documentación con todo detalle. Los programas prueban casi todas las contingencias especiales que podrían resultar en errores o fallas. Al final de cada capítulo analizaremos algunos de los paquetes de propósito general adecuados.

Los paquetes comercialmente disponibles también representan los métodos numéricos de vanguardia. A menudo, su contenido está basado en los paquetes de dominio público, pero incluyen métodos en bibliotecas para casi cualquier tipo de problemas.

Las IMSL (International Mathematical and Statistical Libraries; Bibliotecas Estadísticas y Matemáticas Internacionales) están formadas por bibliotecas MATH, STAT y SFUN para matemáticas numéricas, estadísticas y funciones especiales, respectivamente. Estas bibliotecas contienen más de 900 subrutinas originalmente disponibles en FORTRAN 77 y ahora disponibles en C, FORTRAN90 y JAVA. Estas subrutinas resuelven los problemas de análisis numéricos más comunes. Las librerías están comercialmente disponibles en Visual Numerics.

Los paquetes se entregan en formato compilado con documentación amplia. Existe un programa de ejemplo para cada rutina, así como información de referencia de fondo. IMSL contiene métodos para sistemas lineales, análisis de sistemas propios, interpolación y aproximación, integración y diferenciación, ecuaciones diferenciales, transformadas, ecuaciones no lineales, optimización y operaciones básicas matriz/vector. La biblioteca también incluye amplias rutinas estadísticas.

El Numerical Algorithms Group (NAG) se fundó en Reino Unido en 1971 y desarrolló la primera biblioteca de software matemático. Actualmente tiene más de 10000 usuarios a nivel mundial y contiene más de 1000 funciones matemáticas y estadísticas que van desde software de simulación estadística, simbólica, visualización y numérica hasta compiladores y herramientas de desarrollo de aplicaciones.

Originalmente, MATLAB se escribió para proporcionar acceso al software de matriz desarrollado en proyectos LINPACK y EISPACK. La primera versión se escribió a finales de la década de 1970 para utilizarse en cursos de teoría de matriz, álgebra lineal y análisis numérico. Actualmente, existen más de 500 000 usuarios de MATLAB en más de 100 países.

Las rutinas NAG son compatibles con Maple, desde la versión 9.0.

El Numerical Algorithms Group (NAG; Grupo de Algoritmos Numéricos) ha existido en Reino Unido desde 1970. NAG ofrece más de 1000 subrutinas en una biblioteca FORTRAN 77, aproximadamente 400 subrutinas en una biblioteca C, más de 200 subrutinas en una biblioteca FORTRAN 90 y una biblioteca MPI FORTRAN para máquinas paralelas y agrupaciones de estaciones de trabajo o computadoras personales. Una introducción útil para las rutinas NAG es [Ph]. La biblioteca NAG contiene rutinas para realizar la mayor parte de las tareas de análisis numérico estándar de manera similar a la de IMSL. También incluye algunas rutinas estadísticas y un conjunto de rutinas gráficas.

Los paquetes IMSL y NAG están diseñados para los matemáticos, científicos o ingenieros que desean llamar subrutinas de alta calidad de C, Java o FORTRAN desde dentro de un programa. La documentación disponible con los paquetes comerciales ilustra el programa activador común, requerido para utilizar las rutinas de la librería. Los siguientes tres paquetes de software son ambientes autónomos. Cuando se activan, los usuarios introducen comandos para hacer que el paquete resuelva un problema. Sin embargo, cada paquete permite programar dentro del lenguaje de comando.

MATLAB es una matriz de laboratorio que, originalmente, era un programa Fortran publicado por Cleve Moler [Mo] en la década de 1980. El laboratorio está basado principalmente en las subrutinas EISPACK y LINPACK, a pesar de que se han integrado funciones, como sistemas no lineales, integración numérica, splines cúbicos, ajuste de curvas, optimización, ecuaciones diferenciales normales y herramientas gráficas. Actualmente, MATLAB está escrito en C y lenguaje ensamblador y la versión para PC de este paquete requiere un coprocesador numérico. La estructura básica es realizar operaciones de matriz, como encontrar los valores propios de una matriz introducida desde la línea de comando o desde un archivo externo a través de llamadas a funciones. Éste es un sistema autónomo poderoso que es especialmente útil para instrucción en un curso de álgebra lineal aplicada.

El segundo paquete es GAUSS, un sistema matemático y estadístico producido por Lee E. Ediefson y Samuel D. Jones en 1985. Está codificado principalmente en lenguaje ensamblador y basado EISPACK y LINPACK. Al igual que en el caso de MATLAB, existe integración/diferenciación, sistemas no lineales, transformadas rápidas de Fourier y gráficas. GAUSS se orienta menos hacia la enseñanza de álgebra lineal y más hacia el análisis estadístico de datos. Este paquete también utiliza un coprocesador numérico, cuando existe uno.

El tercer paquete es Maple, un sistema de álgebra computacional desarrollado en 1980. Mediante el Symbolic Computational Group (Grupo Computacional Simbólico) en la University of Waterloo. El diseño para el sistema original Maple se presenta en el artículo de B.W. Char, K.O. Geddes, W.M. Gentlemen y G.H. Gonnet [CGGG].

Maple, escrito en C, tiene la capacidad de manipular información de manera simbólica. Esta manipulación simbólica permite al usuario obtener respuestas exactas, en lugar de valores numéricos. Maple puede proporcionar respuestas exactas a problemas matemáticos como integrales, ecuaciones diferenciales y sistemas lineales. Contiene una estructura de programación y permite texto, así como comandos, para guardarlos en sus archivos de hojas de trabajo. Estas hojas de trabajo se pueden cargar a Maple y ejecutar los comandos.

El igualmente popular Mathematica, liberado en 1988, es similar a Maple.

Existen numerosos paquetes que se pueden clasificar como paquetes de supercalculadora para la PC. Sin embargo, éstos no deberían confundirse con el software de propósito general que se ha mencionado aquí. Si está interesado en uno de estos paquetes, debería leer *Supercomputers on the PC (Supercalculadoras en la PC)* de B. Simon y R. M. Wilson [SW].

Información adicional sobre el software y las bibliotecas de software se puede encontrar en los libros de Cody y Waite [CW] y de Kockler [Ko] y el artículo de 1995 de Dongarra y Walker [DW]. Más información sobre cálculo de punto flotante se puede encontrar en el libro de Chaitini-Chatelin y Frayse [CF] y el artículo de Goldberg [Go].

Los libros que abordan la aplicación de técnicas numéricas sobre computadoras paralelas incluyen los de Schendell [Sche], Phillips and Freeman [PF] y Golub y Ortega [GO].